

Développement Web

Prof. Houda Mouttalib

Année: 2025-2026

3IIR

AGENDA

Rappel au développement web	1
Introduction à JavaScript	2
Programmation avancée en JavaScript	3
Introduction à TypeScript	4
Frameworks et bibliothèques JavaScript	5
Développement d'applications web avec TypeScript	6

Objectifs

- Saisir les bases fondamentales de la technologie Web.
- Maîtriser HTML pour créer des sites web sémantiquement riches.
- Concevoir des mises en page web modernes et responsives avec CSS.
- Déverrouiller la puissance de JavaScript pour des applications web interactives.

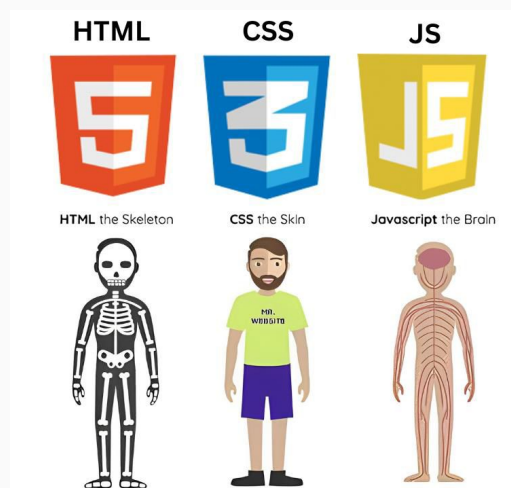


3

Pourquoi apprendre HTML, CSS, et Javascript?

- HTML : La colonne vertébrale de chaque page web, il définit la structure et le contenu.
- CSS : Les pages web avec des styles, des mises en page et des designs responsives.
- JavaScript : Ajoute de l'interactivité, améliore l'expérience utilisateur et permet le contenu dynamique.

À noter : HTML et CSS ne sont pas considérés comme des langages de programmation car ils n'impliquent pas d'opérations logiques telles que les conditions, les boucles ou les fonctions.



4

Outils et logiciels nécessaires

Éditeur de code : Visual Studio Code, Sublime Text, etc.

Navigateurs : Chrome, Firefox...

Contrôle de version : Git/GitHub pour la gestion de projets (optionnel).



5

Comprendre le web

Le web est un système interconnecté d'informations où les personnes et les appareils communiquent et partagent du contenu en utilisant un réseau mondial. C'est l'espace où vivent les sites web, nous permettant d'accéder et d'interagir avec l'information à tout moment et en tout lieu.

Tim Berners-Lee a inventé le World Wide Web en 1989. À l'origine, le web a été développé pour répondre au besoin de partage d'informations entre les scientifiques travaillant dans différentes universités à travers le monde.

The Evolution of Web



Web 1
read-only web



Web 2
read-write web

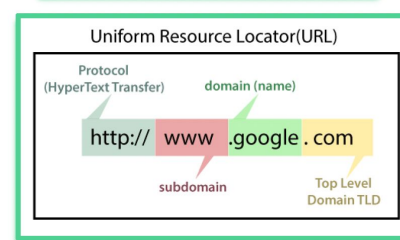
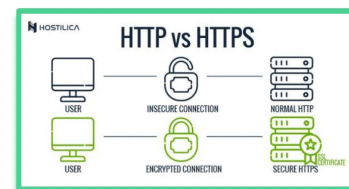
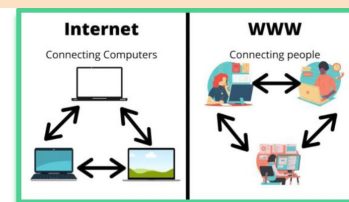


Web 3
read-write-execute web

6

Comprendre le web

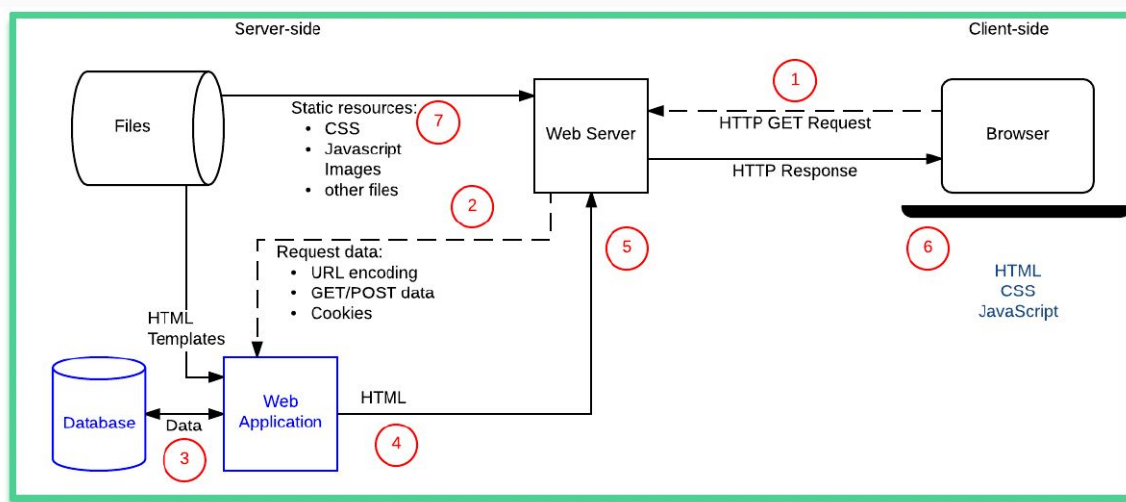
- ❑ Internet vs. World Wide Web :
 - Internet est l'infrastructure ; le web est un service construit dessus.
- ❑ HTTP et HTTPS :
 - Protocoles pour transmettre des données de manière sécurisée entre clients (navigateurs) et serveurs.
- ❑ Noms de domaine :
 - Adresses lisibles par l'homme (par exemple, www.example.com) traduites en adresses IP via le DNS.



7

- Le client (navigateur) envoie une requête HTTP lorsque vous visitez un site web.
- Le serveur traite la requête et renvoie les ressources (HTML, CSS, JS).

Exemple : Taper une URL et voir comment le navigateur récupère et affiche la page web.



8

Les bases de l'HTML

HTML(HyperText Markup Language), ou langage de balisage hypertexte, est un langage simple utilisé pour créer et structurer le contenu des pages web. Il permet de définir la structure de textes, images, liens, et autres éléments.

Qu'est-ce que les balises et éléments HTML ?

- Balises HTML : Les balises sont des codes qui indiquent au navigateur comment afficher le contenu. Elles sont écrites entre des chevrons, par exemple : `<p>`, `<a>`, `<div>`.
- Éléments HTML : Les éléments sont le code complet allant de la balise ouvrante à la balise fermante, incluant le contenu, par exemple : `<p>Ceci est un paragraphe.</p>`

Les deux types de balises HTML : balises paires (conteneur), balises auto-fermantes (vides).

```
<opening-tag> content here </closing-tag>
```

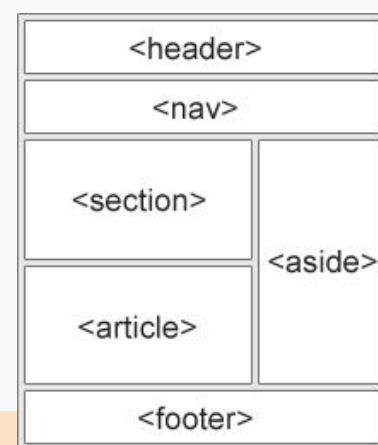
```
<self-closing/>
```

Pourquoi organise-t-on le contenu?

Une bonne organisation améliore la lisibilité, la maintenabilité, l'expérience utilisateur et le référencement (SEO).

Structure de base d'un document HTML :

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Webpage</title>
  </head>
  <body>
    <header>...</header>
    <main>...</main>
    <footer>...</footer>
  </body>
</html>
```



Structure du document html

1. **<!DOCTYPE>** : aide le navigateur à comprendre quelle version de HTML est utilisée afin qu'il puisse afficher correctement la page.
2. **<html>** : L'élément racine qui contient tous les autres éléments d'un document HTML. Il indique au navigateur que le document est un fichier HTML.
3. **<head>** : Contient les métadonnées du document, comme son titre, des liens externes (feuilles de style, scripts...).
4. **<title>** : Définit le titre de la page web, qui apparaît dans l'onglet du navigateur.
5. **<body>** : Contient le contenu principal du document HTML visible par les utilisateurs.
6. **<header>** : Représente le contenu introductif ou un groupe de liens de navigation. Peut servir pour l'en-tête de page ou de section.
7. **<nav>** : Contient les liens de navigation du document, comme un menu ou une table des matières.
8. **<main>** : Désigne le contenu principal du document. Il ne doit y avoir qu'un seul <main> par page.

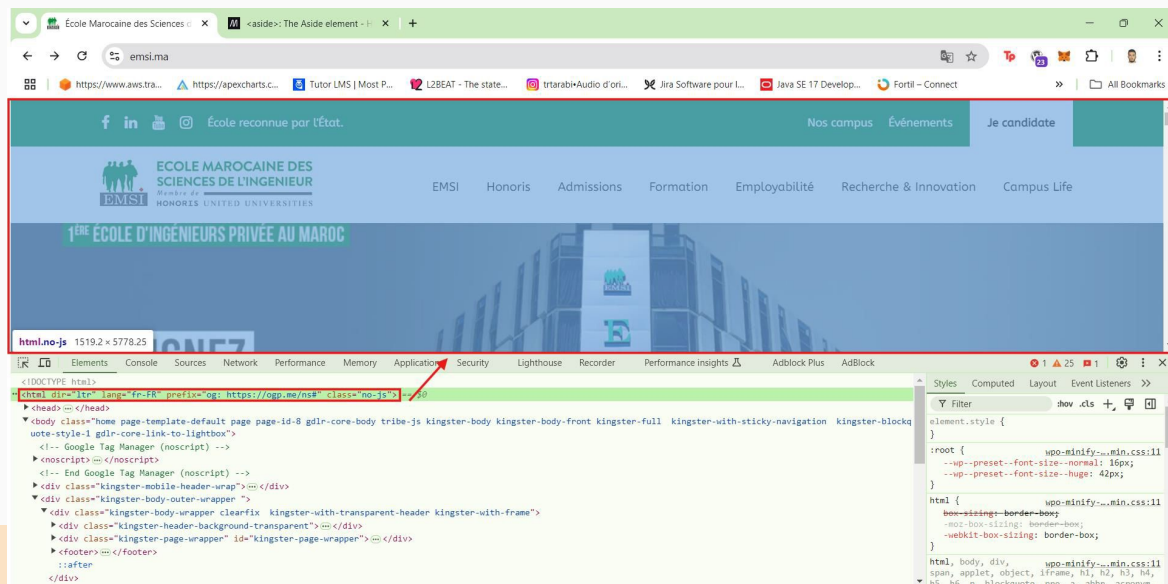
11

Structure du document html

9. **<section>** : Utilisé pour définir des sections dans un document, comme des chapitres ou des parties d'une page web.
10. **<article>** : L'élément HTML représente une composition autonome dans un document, une page, une application ou un site, destinée à être distribuée ou ré-utilisée indépendamment.
11. **<aside>** : Représente un contenu indirectement lié au contenu principal, comme une boîte d'appel, des publicités.
12. **<div>** : Un élément de niveau bloc en HTML utilisé pour regrouper du contenu lié. Il n'a pas de signification particulière ou d'apparence visuelle propre, mais permet aux développeurs d'appliquer des styles, des mises en page et des scripts à une section de contenu.
13. **<footer>** : Représente le pied de page ou de section, contenant typiquement des informations comme le copyright, les contacts ou les liens.

12

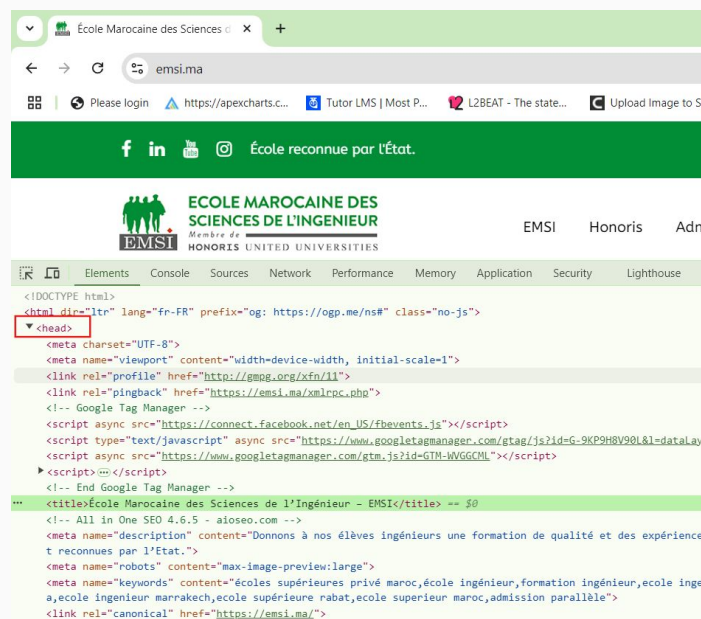
<html> : L'élément racine qui contient tous les autres éléments d'un document HTML. Il indique au navigateur que le document est un fichier HTML



13

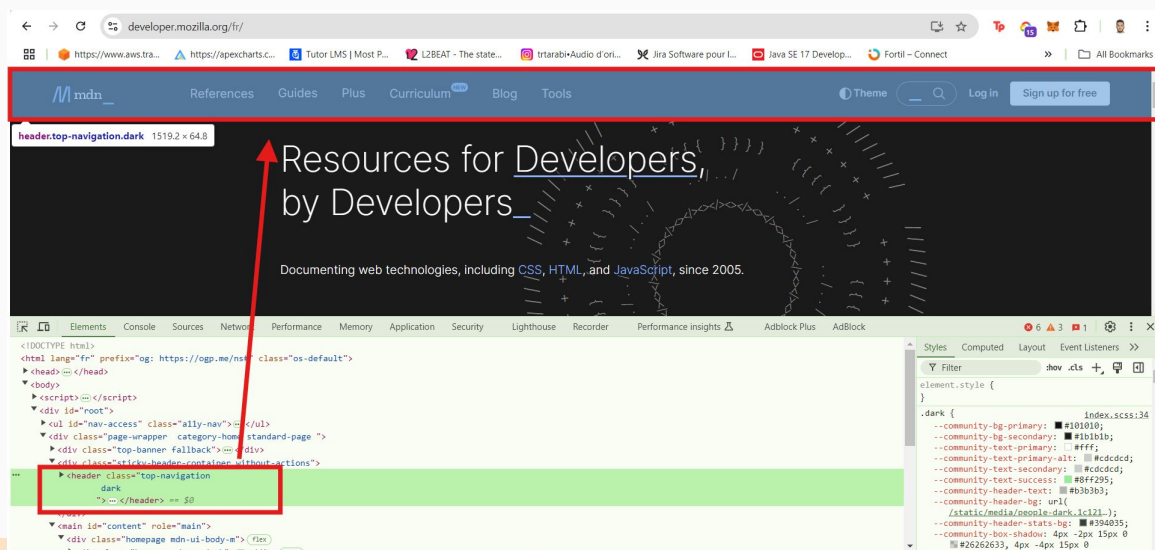
<head>:

1. **<meta charset="UTF-8">**: encodage des caractères pour le document.
2. **<meta name="viewport" content="width=device-width, initial-scale=1">**: garantit que la page est réactive, ce qui signifie qu'elle ajuste sa mise en page et sa taille en fonction de la largeur d'écran de l'appareil.
3. **Autres éléments** :
 - a. Métadonnées : inclut les balises <meta> pour la description et les mots-clés, améliorant le référencement (SEO)
 - b. Feuilles de style : liens vers des fichiers CSS externes qui définissent l'apparence visuelle et la mise en page de la page web.
 - c. Scripts : références à des fichiers JavaScript externes, comme des outils d'analyse (par exemple, Google Tag Manager, Facebook Pixel).



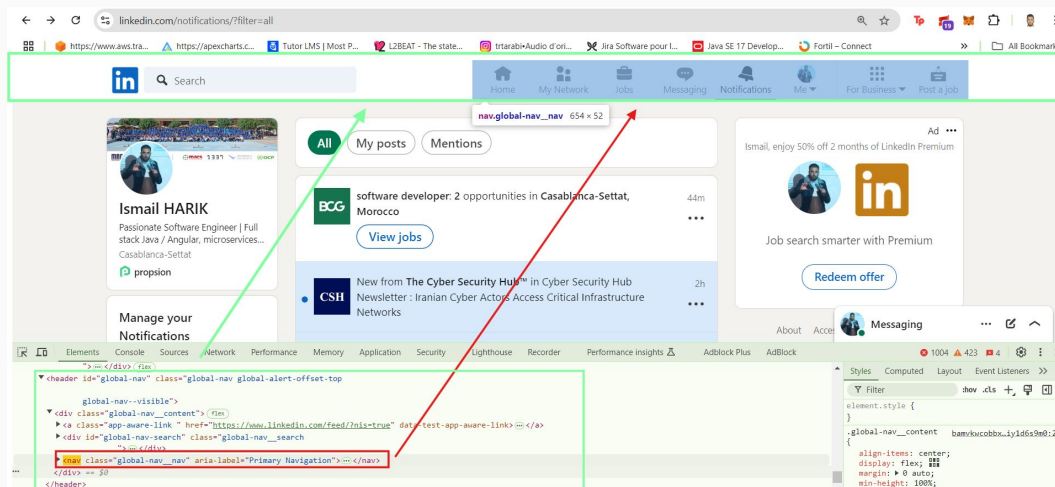
14

<header>: Représente le contenu introductif ou un groupe de liens de navigation. Peut être utilisé pour les en-têtes de page ou les en-têtes de section.



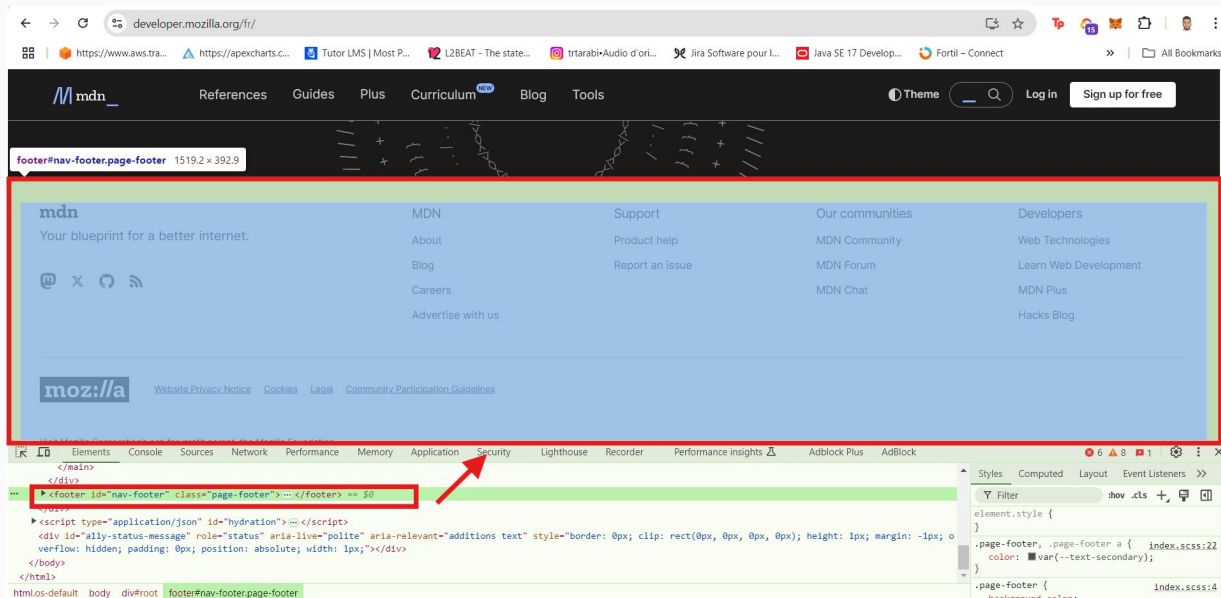
15

<nav>: Contient des liens de navigation pour le document, comme un menu ou une table des matières.



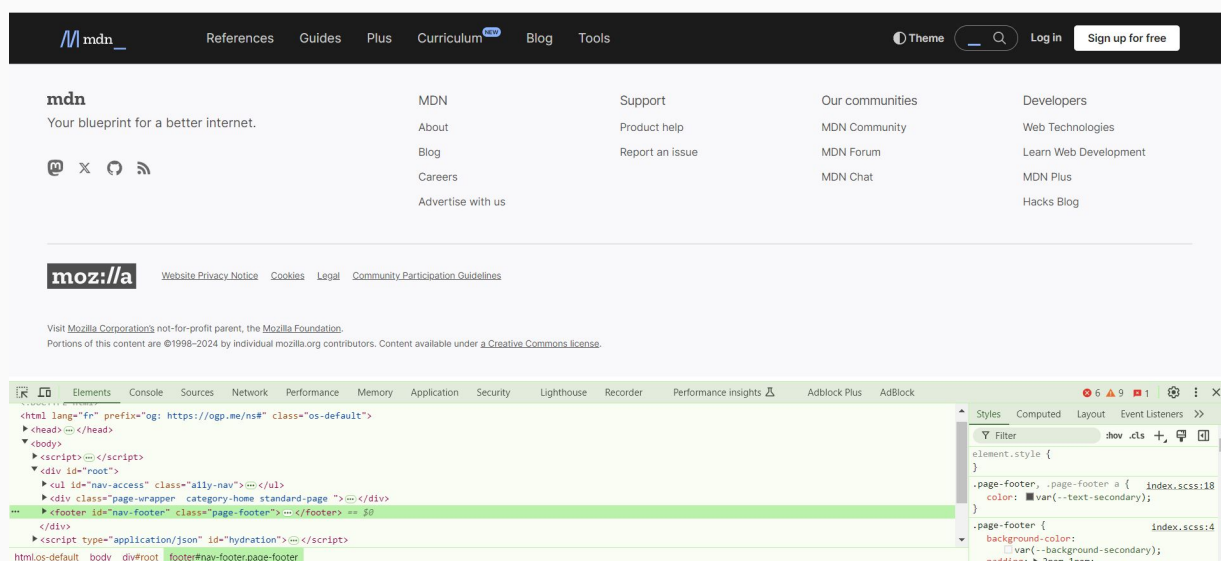
10

<footer>: Représente le contenu du pied de page d'une page ou section, contenant généralement des informations telles que les droits d'auteur, les coordonnées ou les liens.



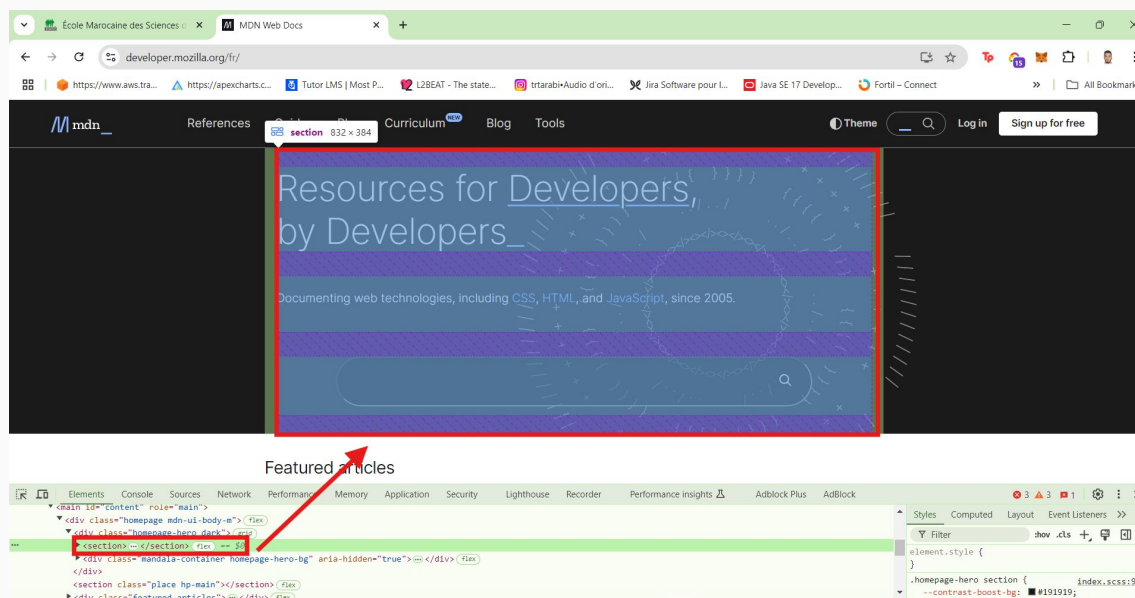
17

Comme vous pouvez le voir, après avoir supprimé l'élément **<main>** du **<body>**, seules la barre de navigation et l'en-tête sont restés visibles sur la page.



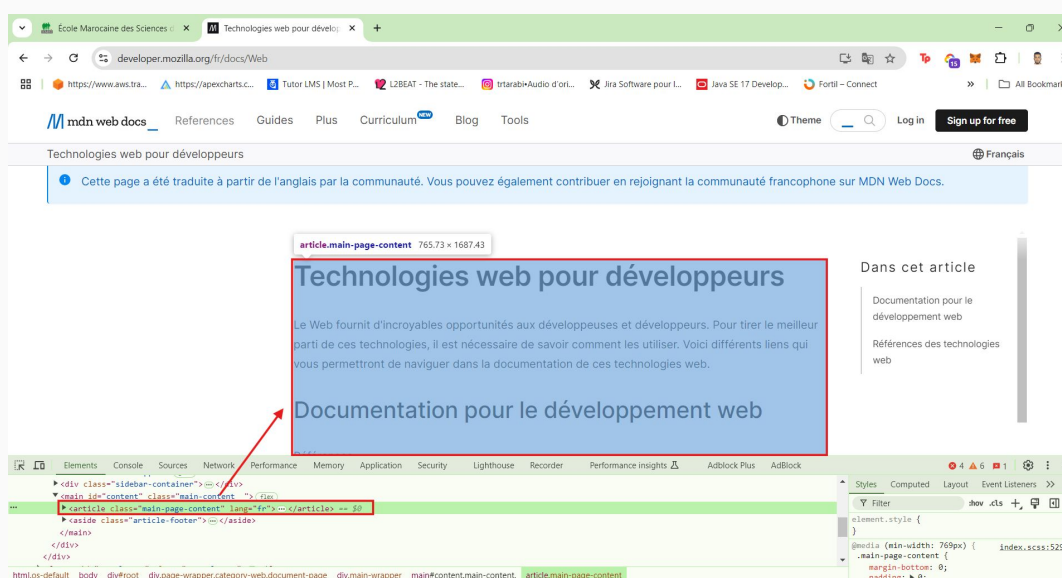
18

<section>: Utilisé pour définir des sections dans un document, telles que des chapitres ou des parties d'une page web.



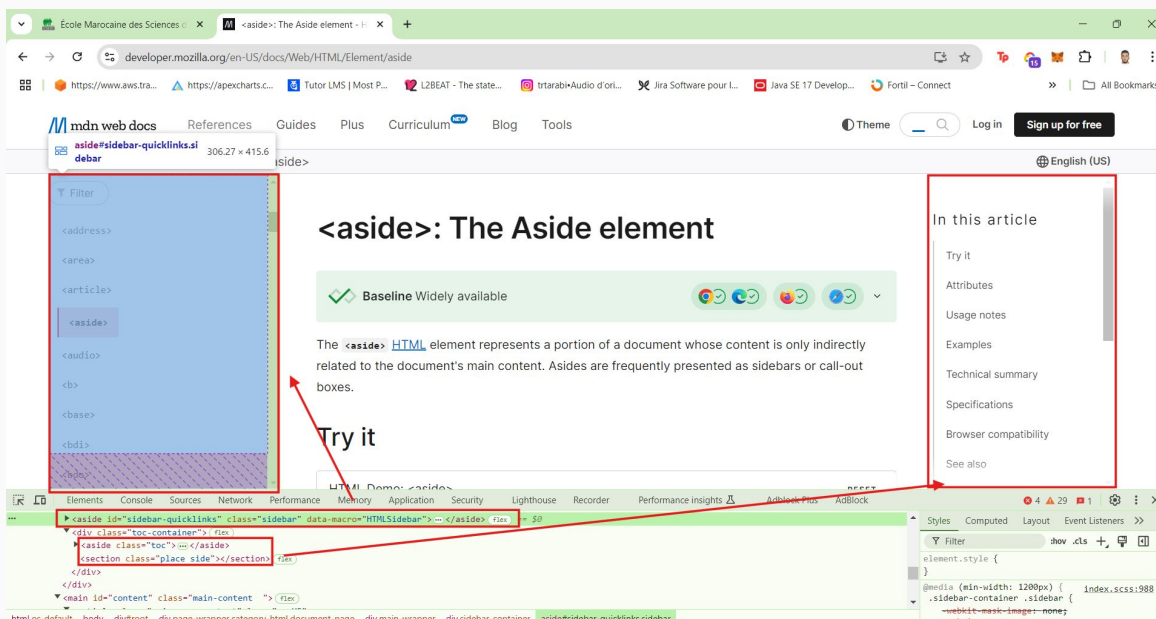
19

<article>: L'élément HTML représente une composition autonome dans un document, une page, une application ou un site, destinée à être distribuée ou réutilisable de manière indépendante.



20

<aside>: Représente le contenu qui est indirectement lié au contenu principal, comme une barre latérale, un encadré ou des publicités.



21

Mise en forme et style du texte :

<p>: Définit un paragraphe de text

<h1> to <h6>: Les balises de titre utilisées pour créer des titres, <h1> étant la plus importante et <h6> la moins.

**** : Représente une forte importance et rend le texte en gras.

****: Représente le texte souligné et le rend généralement en italique.

****: Met le texte en gras mais n'ajoute pas d'importance supplémentaire comme .

<i> : Met le texte en italique mais n'ajoute pas d'emphasis comme .

<small> : Affiche du texte plus petit, souvent utilisé pour les avertissements ou les notes de côté.

<sub> : Représente un texte en indice (par exemple, H₂O).

<sup> : Représente le texte en exposant (par exemple, E=mc²).

<abbr> : Définit une abréviation ou un acronyme, avec un attribut title pour la description complète (par exemple, <abbr title="HyperText Markup Language">HTML</abbr>).

<blockquote> : Définit un long bloc de texte cité, souvent en retrait.

<code> : Définit un fragment de code informatique.

<pre> : garder les espaces à l'intérieur de la balise de code

22

liens et hyperliens:

<a> : Crée un lien hypertexte qui peut mener à une autre page, fichier ou emplacement dans le même document. L'attribut href spécifie l'URL.

- **href** : Spécifie l'URL de destination.

- **target** : Spécifie où ouvrir le document lié (**_blank** pour un nouvel onglet, **_self** sur le même tab).

- **download** : Spécifie que le lien doit télécharger le fichier cible plutôt que d'y naviguer.

- **mailto:** : Crée un lien qui ouvre le client de messagerie par défaut pour envoyer un email.

- **tel** : Crée un lien pour appeler un numéro de téléphone sur les appareils pris en charge.

```
<a href="https://www.emsi.ma">link to emsi's website</a><br>
<a href="tel:+212522894287">Tel: 0522 89 42 87</a><br>
<a href="mailto:info.casa@emsi.ma">Email: info.casa@emsi.ma</a><br>
<a href="Public/docs/Plaqueette-EMSI-2025.pdf" download>Brochure</a>
```

23

Listes et navigation :

**** : Crée une liste non ordonnée avec des puces.

**** : Crée une liste ordonnée avec des chiffres ou des lettres.

**** : Représente un élément dans une liste, utilisé au sein de **** ou ****.

<dl> : Définit une liste de description pour les termes et les descriptions.

<dt> : Représente un terme dans une liste de description.

<dd> : Représente la description du terme dans une liste de descriptions.

```
<ul>
  <li>Apple</li>
  <li>Orange</li>
</ul>

<ol>
  <li>Apple</li>
  <li>Orange</li>
</ol>

<dl>
  <dt>Apple</dt>
  <dd>fruit can be green, red, or yellow.</dd>

  <dt>Orange</dt>
  <dd>fruit always orange.</dd>
</dl>
```

24

Section One

Article 1 Links

[Visitez Notre Ecole \(SELF\)](#)

[Visitez Notre Ecole \(BLANK\)](#)

[Call Us](#)

[Send An Email](#)

End Article 1

Article 2 Lists

Lists

OL: Order List

4. element 1
5. element 2
6. element 3

UL: Unordered List

- default element 1
- default element 2
- default element 3
- square element 1
- square element 2
- square element 3

End Article 2

End Section One

Section Two Content

Footer Content © 2024 - All Rights Reserved

Exercice TP 1:

Section One

Article 1 Links

[Visitez Notre Ecole \(SELF\)](#)

[Visitez Notre Ecole \(BLANK\)](#)

[Call Us](#)

[Send An Email](#)

End Article 1

Article 2 Lists

Lists

OL: Order List

4. element 1
5. element 2
6. element 3

UL: Unordered List

- default element 1
- default element 2
- default element 3
- square element 1
- square element 2
- square element 3

End Article 2

Section Two Content

Footer Content © 2024 - All Rights Reserved

éléments multimédias:

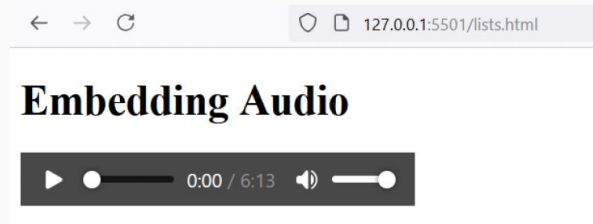
**** : Intègre une image dans le document. Nécessite les attributs **src** (source) et **alt** (texte alternatif).

<audio> : Intègre du contenu audio. Nécessite des éléments **<source>** pour spécifier les fichiers audio et peut avoir des contrôles pour activer la lecture/pause.

<video> : Intègre du contenu vidéo. Nécessite des éléments **<source>** et peut avoir des attributs comme les contrôles, le mode automatique, la largeur et la hauteur.

<iframe> : Intègre un autre document HTML dans le document actuel, comme une vidéo ou une page web.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Embedding Audio Example</title>
</head>
<body>
  <h1>Embedding Audio</h1>
  <audio controls>
    <source
      src="https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3"
      type="audio/mpeg">
    Your browser does not support the audio element.
  </audio>
</body>
</html>
```



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1>Embedding Video</h1>
  <video controls>
    <source src="https://www.w3schools.com/html/mov_bbb.mp4" type="video/mp4">
    Your browser does not support the video tag.
  </video>
</body>
</html>
```

Embedding Video



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Embedding Iframe Example</title>
</head>
<body>
  <h1>Embedding Iframe</h1>
  <iframe width="600" height="400" src="https://www.emsi.ma" title="EMSI Website"></iframe>
</body>
</html>
```

Embedding Iframe



Tableaux et présentation des données:

<table> : Crée une table pour organiser les données en lignes et colonnes.

<tr> : Définit une ligne dans une table.

<td> : Définit une cellule (données) dans une ligne de tableau.

<th> : Définit une cellule d'en-tête dans un tableau, généralement affichée en gras.

<thead> : Groupe le contenu de l'en-tête dans une table.

<tbody> : Groupe le contenu du corps dans une table.

<tfoot> : Contenu du pied de page des groupes dans un tableau.

<caption> : Ajoute un titre ou une légende à un tableau.

colspan : Rend une cellule étendue sur plusieurs colonnes.

rowspan : Rend une cellule étendue sur plusieurs lignes.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Basic Table Example</title>
</head>
<body>
  <h1>Basic Table</h1>
  <table border="1">
    <tr>
      <th>Name</th>
      <th>Age</th>
      <th>Country</th>
    </tr>
    <tr>
      <td>Alice</td>
      <td>30</td>
      <td>USA</td>
    </tr>
    <tr>
      <td>Bob</td>
      <td>25</td>
      <td>UK</td>
    </tr>
  </table>
</body>
</html>
```

Basic Table

Name	Age	Country
Alice	30	USA
Bob	25	UK

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Table with Caption Example</title>
</head>
<body>
  <h1>Table with Caption</h1>
  <table border="1" width="300">
    <caption>Team Members</caption>
    <tr>
      <th>Name</th>
      <th>Role</th>
    </tr>
    <tr>
      <td>Sarah</td>
      <td>Developer</td>
    </tr>
    <tr>
      <td>John</td>
      <td>Designer</td>
    </tr>
  </table>
</body>
</html>
```

Table with Caption

Team Members

Name	Role
Sarah	Developer
John	Designer

33

```
<table border="1" width="300px">
  <thead>
    <tr>
      <th>Product</th>
      <th>Price</th>
      <th>Quantity</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Apple</td>
      <td>$1.00</td>
      <td>5</td>
    </tr>
    <tr>
      <td>Banana</td>
      <td>$0.50</td>
      <td>10</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td colspan="2">Total Cost</td>
      <td>$10.00</td>
    </tr>
  </tfoot>
</table>
```

Table with Header, Body, and Footer

Product	Price	Quantity
Apple	\$1.00	5
Banana	\$0.50	10
Total Cost		\$10.00

34


```
<h1>Table with Colspan</h1>
<table border="1" width="300">
  <tr>
    <th colspan="3">Monthly Report</th>
  </tr>
  <tr>
    <td>Sales</td>
    <td>1000</td>
    <td>Units</td>
  </tr>
  <tr>
    <td>Profit</td>
    <td>200</td>
    <td>USD</td>
  </tr>
</table>
```

Table with Colspan

Monthly Report		
Sales	1000	Units
Profit	200	USD

35

Formulaires, attributs et éléments d'entrée:

<form> : Définit un formulaire pour collecter les entrées utilisateur.

- **action** : Spécifie l'URL à laquelle envoyer les données du formulaire lorsqu'elles sont soumises.
- **method** : Spécifie la méthode HTTP (**GET** ou **POST**) pour l'envoi du formulaire.
- **target** : Spécifie où afficher la réponse reçue après la soumission du formulaire.
- **autocomplete** : Spécifie si un formulaire doit avoir l'auto-complétion activée "**on**" ou désactivée "**off**".
- **enctype** : Prend la valeur "**multipart/form-data**" s'il y a des fichiers.

<input> : Permet aux utilisateurs de saisir des données. L'attribut type définit le type d'entrée (par exemple, text, password, radio, checkbox, date, datetime-local, number, tel, range, color,).

<textarea> : Crée un champ de saisie de texte multi-lignes.

<select> : Crée une liste déroulante.

<option> : Représente un élément dans une liste déroulante **<select>**.

<label> : Associe une étiquette de texte avec un contrôle de formulaire pour l'accessibilité.

<button> : Représente un bouton cliquable, souvent utilisé pour soumettre des formulaires.

<fieldset> : Regroupe les éléments liés dans un formulaire.

<legend> : Fournit une légende pour le **<fieldset>**.

36

Exemple:

```
<form action="" method="post" target="_blank">
  <label for="name">Name:</label>
  <input type="text" id="name" name="username">
  <button type="submit">OK</button>
</form>
```

37

Exemple:

```
<label for="password">Password:</label>
<input type="password" id="password" name="userpassword"><br>

<label for="email">Email:</label>
<input type="email" id="email" name="useremail"><br>
```

38

Exemple:

Gender:

☐ Male ☐ Female

Interests:

☐ Coding ☐ Sports

39

```
<p>Gender:</p>
```

```
<input type="radio" id="male" name="gender" value="male">
```

```
<label for="male">Male</label>
```

```
<input type="radio" id="female" name="gender" value="female">
```

```
<label for="female">Female</label>
```

```
<p>Interests:</p>
```

```
<input type="checkbox" id="coding" name="interest" value="coding">
```

```
<label for="coding">Coding</label>
```

```
<input type="checkbox" id="sports" name="interest" value="sports">
```

```
<label for="sports">Sports</label><br><br><br>
```

40

Exemple:

Birth Date: 

Age:

Favorite Color:

41

```
<label for="birthdate">Birth Date:</label>
<input type="date" id="birthdate" name="birthdate"><br>

<label for="age">Age:</label>
<input type="number" id="age" name="age" min="1" max="99"><br>

<label for="color">Favorite Color:</label>
<input type="color" id="color" name="color"><br><br><br>
```

42

Exemple:

Upload your CV: No file chosen

Message:

Country:

43

```
<label for="resume">Upload your CV:</label>
<input type="file" id="resume" name="resume"><br><br>
<label for="message">Message:</label>
<textarea id="message" name="message" rows="4" cols="40"></textarea><br><br>
<label for="country">Country:</label>
<select id="country" name="country">
  <option value="">--Select--</option>
  <option value="france">France</option>
  <option value="usa">USA</option>
  <option value="morocco">Morocco</option>
</select><br><br>
```

44

Formulaire d'inscription

1

Informations personnelles

Prénom *

Nom *

Téléphone
ex: +212600000000

Date de naissance *
dd/mm/yyyy

Pays *
--Sélectionner le pays--

Genre *
☐ Homme
☒ Femme

Photo de profil
Choose File No file chosen

Bio courte
Plus d'information...

Exercice TP 2:

Identifiants du compte

2

E-mail du compte *

Mot de passe *

Confirmation *

Préférences & consentements

- ☐ Recevoir des informations et actualités par e-mail
☐ **J'accepte les conditions générales ***

S'inscrire

45

Où placer le CSS ?

- Feuille externe: `<link rel="stylesheet" href="style.css">`
- Interne (dans le HTML):

```
<head>
  <style>
    /* ici */
  </style>
</head>
```

- En ligne (inline): `<p style="color: red;">Texte en rouge</p>`

46

Structure d'une règle CSS

```
sélecteur {
    propriété: valeur;
}
```

Exemple :

```
h1 {
    color: #2152A3;
    text-align: center;
}
```

47

Les principaux sélecteurs CSS:

Sélecteur	Exemple	Cible/Description
Élément	p {}	Tous les éléments<p>
Classe	.important {}	Tous les éléments avec class="important"
Identifiant	#titre {}	L'élément avec id="titre"
Universel	* {}	Tous les éléments
Goupé	h1, h2 {}	Tous les<h1>et<h2>
Descendant	div p {}	Tous les<p>à l'intérieur d'un<div>
Enfant direct	ul > li {}	Les qui sont enfants directs d'un
Frère suivant	h2 + p {}	Le<p>qui suit directement un<h2>

48

Les principaux sélecteurs CSS (suite):

Sélecteur	Exemple	Cible/Description
Frères suivants	<code>h2 ~ p {}</code>	Tous les <code><p></code> qui suivent un <code><h2></code> dans le même parent
Attribut	<code>input[type="text"] {}</code>	Les <code><input></code> avec <code>type="text"</code>
Pseudo-classe	<code>a:hover {}</code>	Les liens <code><a></code> au survol de la souris
Pseudo-élément	<code>p::first-line {}</code>	La première ligne de chaque <code><p></code>
N-ième enfant	<code>tr:nth-child(2) {}</code>	Le deuxième <code><tr></code> dans son parent (tableau)
N-ième type	<code>li:nth-of-type(3) {}</code>	Le troisième <code></code> de son type dans le parent
Premier enfant	<code>p:first-child {}</code>	Le <code><p></code> qui est le premier enfant de son parent
Dernier enfant	<code>p:last-child {}</code>	Le <code><p></code> qui est le dernier enfant de son parent

49

Les valeurs de couleur en CSS:

- On peut utiliser des noms prédéfinis en anglais comme : *red, orange, green, white, etc ..*
- Une couleur hexadécimale commence par # et contient 6 chiffres/lettres (de 0 à F) :
 - `#FF0000` = rouge
 - `#00FF00` = vert
 - `#0000FF` = bleu
 - On peut aussi utiliser la version courte : `#F00` (pour `#FF0000`).
- RGB signifie Red Green Blue : chaque composant va de 0 à 255:
 - `rgb(255, 0, 0)` = rouge
 - `rgb(0, 255, 0)` = vert
 - `rgb(0, 0, 255)` = bleu
- Avec RGBA, on ajoute la transparence (alpha, de 0 à 1) :
 - `rgba(255, 0, 0, 0.5)` = rouge à 50% de transparence

50

Les unités de mesure en CSS:

Unité	Signification	Exemple	Utilisation courante
px	Pixel (valeur fixe)	width: 100px;	Largeur, hauteur, bordure, police
%	Pourcentage du parent	width: 50%;	Largeur, hauteur, marges, padding
em	Taille relative à l'élément parent	font-size: 2em;	Police, marges, padding
rem	Taille relative à l'élément racine	font-size: 1.5rem;	Police, marges, padding
vw	Pourcentage de la largeur de la fenêtre	width: 80vw;	Largeur responsive
vh	Pourcentage de la hauteur de la fenêtre	height: 50vh;	Hauteur responsive

51

Les unités de mesure en CSS: exemples

```
.parent {
  font-size: 20px;
}
.enfant {
  font-size: 2em; /* 2 × 20px = 40px */
}
```

```
html {
  font-size: 16px;
}
.titre {
  font-size: 2rem; /* 2 × 16px = 32px */
}
```

60 point serif
54 point serif
48 point serif
36 point serif
30 point serif
26 point serif
24 point serif
22 point serif
20 point serif
18 point serif
14 point serif
12 point serif
10 point serif
8 point serif
6 point serif
4 point serif
2 point serif

60 point sans serif
54 point sans serif
48 point sans serif
36 point sans serif
30 point sans serif
26 point sans serif
24 point sans serif
22 point sans serif
20 point sans serif
18 point sans serif
14 point sans serif
12 point sans serif
10 point sans serif
8 point sans serif
6 point sans serif
4 point sans serif
2 point sans serif

52

Propriétés CSS essentielles et leurs valeurs

Propriété	Rôle	Valeurs possibles et exemples
color	Couleur du texte	Noms (red,blue), HEX (#2152A3), RGB (rgb(33,82,163)), HSL (hsl(215,60%,45%))
background-color	Couleur de fond	Même valeurs que color
margin	Espace extérieur autour de l'élément	Unité : px, em, %, auto. Ex :margin: 20px;ou margin: 10px 5px;(haut/bas, gauche/droite)
padding	Espace intérieur (autour du contenu)	Unité : px, em, %. Ex :padding: 10px;ou padding: 5px 15px;
width	Largeur de l'élément	px, %, em, auto. Ex :width: 200px;ou width: 80%;
height	Hauteur de l'élément	px, %, em, auto. Ex :height: 50px;

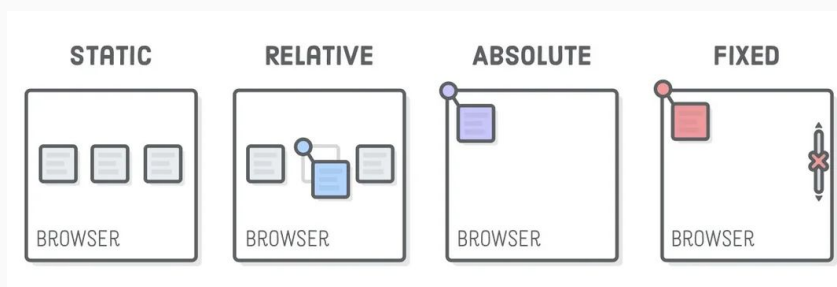
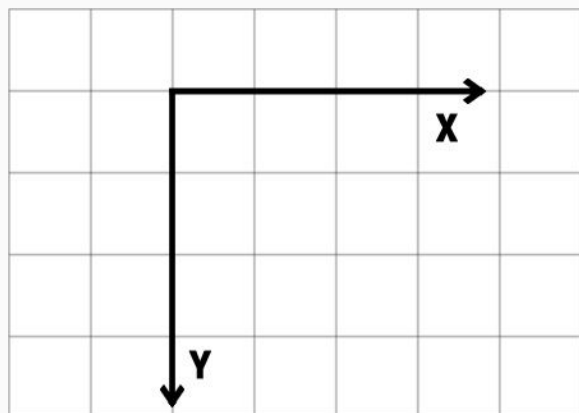
53

Propriétés CSS essentielles et leurs valeurs

Propriété	Rôle	Valeurs possibles et exemples
border	Bordure autour de l'élément	Largeur (px), style (solid,dashed,dotted), couleur. Ex :border: 1px solid #ccc;
border-radius	Coins arrondis	px, %. Ex :border-radius: 8px;ou border-radius: 50%;(cercle)
font-size	Taille du texte	px, em, rem, %. Ex :font-size: 18px;ou font-size: 1.2em;
font-family	Police du texte	Nom(s) de police. Ex :font-family: Arial, sans-serif;
text-align	Alignement du texte	left,right,center,justify. Ex :text-align: center;
box-shadow	Ombre portée	Décalage horizontal, vertical, flou, couleur. Ex :box-shadow: 0 2px 6px #aaa;
display	Type d'affichage	block,inline,inline-block,flex,none, etc. Ex :display: flex;

54

Propriétés CSS essentielles et leurs valeurs: position



55

Utilisation des Media Queries en CSS:

Les media queries sont des règles CSS qui permettent d'appliquer certains styles uniquement selon les caractéristiques du navigateur ou de l'appareil (largeur, orientation, résolution...).

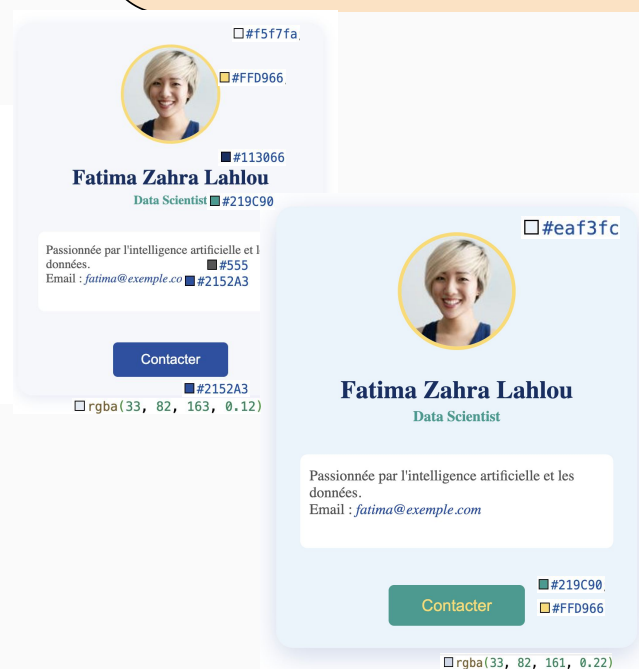
- Media Queries = outils CSS pour personnaliser l'apparence selon l'appareil.
- Responsive = site qui s'adapte à tous les écrans.
- On combine souvent des unités relatives (vw, %, em) et des media pour une vraie flexibilité !

```
@media (max-width: 768px) { /* tablettes */ }
@media (max-width: 480px) { /* smartphone portrait */ }
@media (min-width: 1200px) { /* grands écrans */ }
```

56

Exercice TP 3:

```
<div class="carte-profil">
  
  <h2>Fatima Zahra Lahlou</h2>
  <p class="specialite">Data Scientist</p>
  <p class="presentation">
    Passionnée par l'intelligence artificielle et les données.<br>
    Email : <span class="email">fatima@exemple.com</span>
  </p>
  <a class="contact" href="mailto:fatima@exemple.com">Contacter</a>
</div>
```



57

Framework CSS recommandé:

Bootstrap est un framework CSS open source qui facilite la création de sites web responsives et modernes grâce à des composants et des classes prédéfinis pour le design, la mise en page et l'interactivité, sans avoir à écrire beaucoup de code CSS personnalisé.

Autres frameworks: Tailwind, Materialize et Bulma.

<https://getbootstrap.com/>

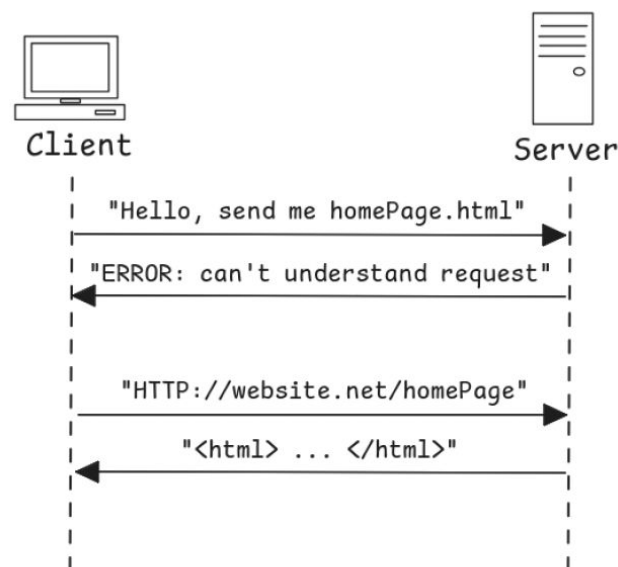


58

1. Introduction:

L'**internet** permet d'échanger plusieurs types de données et services, grâce à des protocoles réseau qui permettent aux clients et aux serveurs de comprendre les requêtes et réponses (HTTP, FTP, DNS, ODBC, SSH, SMTP, VoIP..).

Le **web** est la partie de l'internet qui utilise le protocole HTTP, ou le client est un généralement un navigateur (Chrome, Netscape, Firefox, Edge, Samsung Internet ...), et le serveur est un programme serveur web (Apache HTTP, Nginx, Microsoft IIS, Apache Tomcat ...).



59

1. Introduction:

Architecture client/serveur: L'architecture client-serveur est un modèle où deux parties interagissent :

- Le client est un logiciel (navigateur web, application mail, etc.) qui envoie des requêtes au serveur pour obtenir des services ou des données.
- Le serveur est un logiciel (serveur web, service cloud, etc.) qui répond aux requêtes du client en fournissant les données ou services demandés.

Fonctionnement :

Le client envoie une requête au serveur via un réseau (Internet, réseau local d'un établissement, etc.).

Le serveur traite la requête (récupération de données d'une base, vérification d'authentification, etc.).

Le serveur envoie une réponse au client, qui l'affiche ou l'utilise.

Les termes « client » et « serveur » peuvent aussi désigner le dispositif matériel (ordinateur personnel, machine serveur, smartphone, console, etc.) sur lequel sont installés les logiciels client ou serveur.

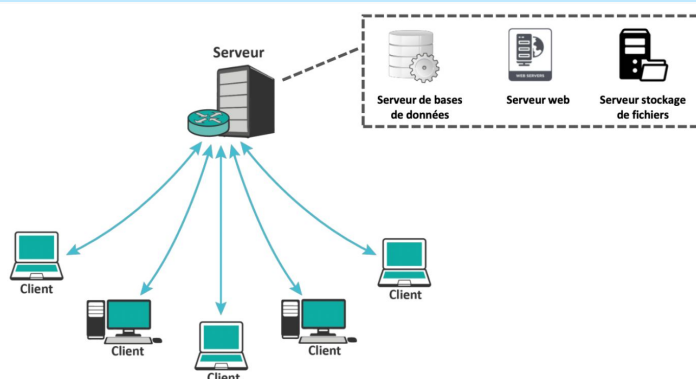


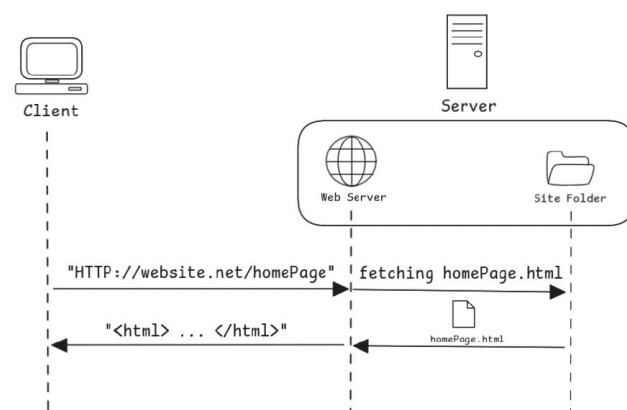
Figure 4 : Architecture Client-serveur [2]

60

1. Introduction:

Pages statiques: Lors de l'invention du web, les pages HTML étaient statiques, stockées en tant que fichiers HTML dans le serveur et prêtes à servir, cela présente quelques limitations en termes d'expérience utilisateur:

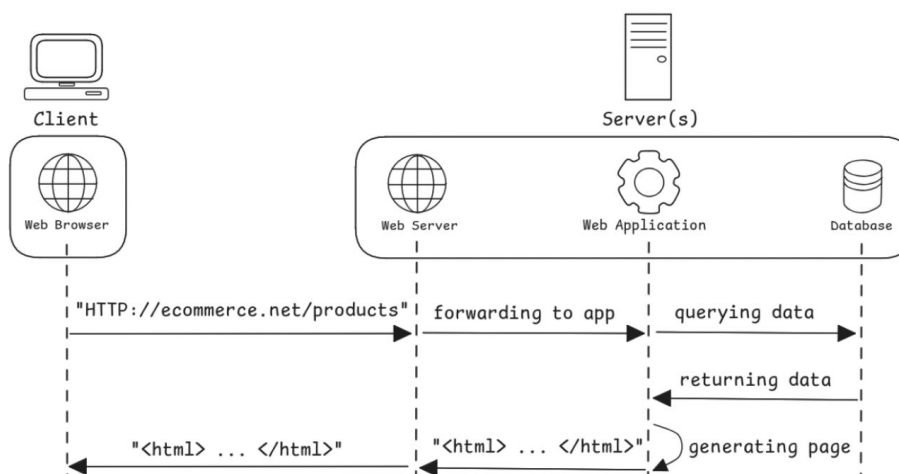
- Le contenu est figé, les données dans la page ne changent pas en fonction des choix de l'utilisateur, sauf si le développeur du site change le code HTML manuellement.
- Les éléments de la page sont inertes, l'utilisateur ne fait que voir la page, il n'y a pas d'interactivité, à l'exception de pouvoir cliquer sur des liens pour naviguer vers d'autres pages.



61

1. Introduction:

Scripting côté serveur: le besoin de dynamisme des pages web vis-à-vis des situations où les données change perpétuellement (eCommerce, forum de discussion ...) a donné naissance à des outils et bibliothèques de programmation web pour différents langages (Perl CGI, PHP, ASP .NET, Java ServerPages, Ruby On Rails ...), ils permettent au serveur de construire une page HTML en se basant sur une source de données dynamique (une BD généralement) à chaque demande du client.



62

1. Introduction:

Scripting coté client: pour ajouter de l'interactivité aux pages HTML, plusieurs langages et outils de programmation spécialisés sont apparus (VB Script, Adobe Flash et ActionScript, Java Applets, **JavaScript** ...), à cause du problème de compatibilité des sites entre navigateurs et grâce aux efforts de standardisation, JavaScript est désormais le seul langage restant, et que comprend tous les navigateurs.

Utilité du scripting coté client: réagir aux événements déclenchés par l'utilisateur (cliques ou survol de souris, saisie dans champs, défilement de page, appui sur touche du clavier ...), pour:

1. Cacher ou faire apparaître des éléments HTML ou les modifier.
2. Valider les champs de formulaire avant envoi au serveur.
3. Optimiser le fonctionnement de la page avec le caching et la délégation de quelques parties du code serveur au côté client.
4. Récupérer des données du serveur ou de services web externes sans recharger la page ou naviguer vers une nouvelle.
5. Adapter la page au contexte de l'utilisateur (navigateur, dimensions de l'écran, date et heure, localisation ...) et à ses préférences (mode claire/sombre, mise en page ...).
6. Créer des animations graphiques et jeux.

1. Introduction:

Le front-end est la partie visible à l'utilisateur, l'interface graphique (UI), le code front-end est la partie du code de l'application web qui est directement responsable de la construction de la page HTML avec les données qui lui sont fournies.

Le back-end désigne la logique fonctionnelle de l'application web, le code back-end est la partie du code qui est chargé de la récupération, modification et traitement des données, il remet des données prêtes à afficher au code front-end.

Attention: Quand on parle de code, il ne faut pas confondre les dualités coté-client / coté-serveur et front-end / back-end, ces premières ont à voir avec le lieu d'exécution du code (machine cliente ou serveur), tandis que ces derniers ont à voir avec ce que le code fait (interface ou fonctionnel), on a donc plusieurs possibilités:

1. Les codes front-end et back-end sont tous deux exécutés côté serveur, le côté client ne fait qu'afficher les pages HTML reçues, cette technique est nommée **«Server-side Rendering (SSR)»** ou rendu côté serveur.
2. Les codes front-end et back-end sont tous deux exécutés côté client, là on est hors web, il n'y a pas de côté serveur, donc on parle plutôt d'applications desktop ou mobile ou autres types d'applications hors ligne.
3. Le code front-end est exécuté côté-client, tandis que le code back-end est exécuté côté serveur, c'est le navigateur qui construit la page HTML avec JavaScript, cette technique est nommée **«Client-side Rendering (CSR)»** ou rendu côté client.
4. Pour les pages statiques en termes de contenu, il y'a un serveur pour servir les pages HTML mais pas de code back-end, puisqu'il n'y a pas de données dynamiques à préparer, les pages sont prêtes en tant que fichiers HTML stockés avant même d'être demandées par un client, cette technique s'appelle **«Pre-Rendering»** ou pré-rendu.

1. Introduction:

Dès la création d'internet en 1991, la partie front-end dans les sites dynamiques était gérée coté serveur (SSR), c'est en l'an 2001 que JavaScript reçoit la fonctionnalité «XML HTTP Request (XHR)» qui permet au client de récupérer des données depuis serveurs, et par conséquent la méthode AJAX fut créée, migrant ainsi du code front-end vers le coté client. En 2006 apparaît «jQuery» qui simplifie la méthode AJAX et popularise la gestion du front-end -partiellement ou entièrement- dans le coté client (CSR).

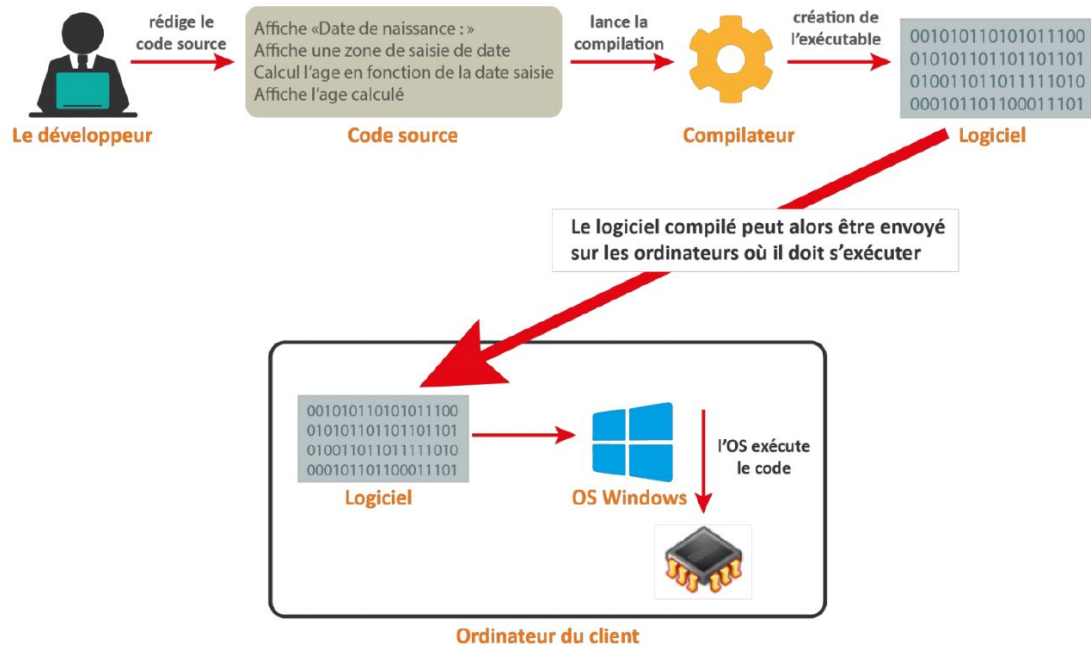
Au début des années 2010 la tendance «Single Page Application (SPA)» gagne du terrain, avec plusieurs bibliothèques et frameworks (Angular.js, Backbone.js, React ...), le principe est de servir au client une seule page HTML statique, souvent vide, dite coquille (shell), suivie du code JavaScript qui contient l'entièreté du code front-end, et qui se chargera de construire le contenu de l'interface et le mettre à jour par la suite, sans aucun rechargement de page, donnant ainsi une expérience utilisateur fluide similaire à celle d'une application.

Attention: Lorsqu'on parle CSR et SSR, il ne faut pas penser en noir et blanc, un même site web peut avoir des pages CSR et d'autre SSR, et même des pages statiques, et une même page peut être construite coté serveur et avoir des éléments modifiés coté client après interaction de l'utilisateur.

1. Introduction:

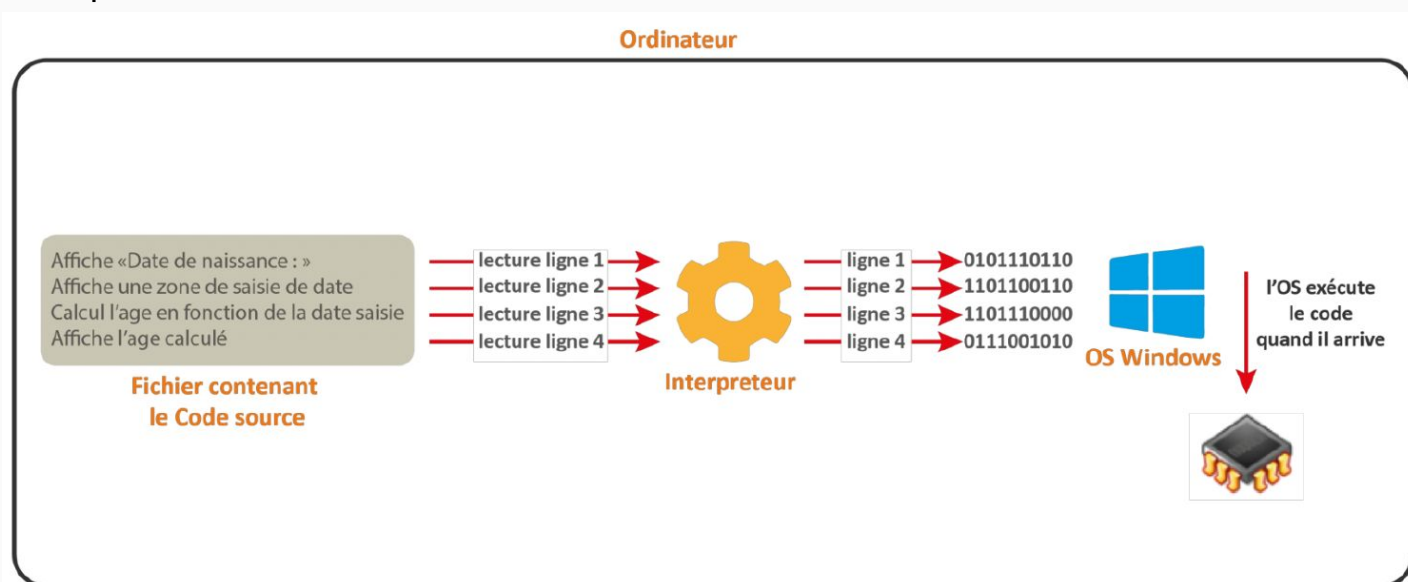
- Langage de programmation créé en 1995 par **Brendan Eich** chez **Netscape**.
 - Fait partie des langages web dits « standards » du web, avec HTML et CSS.
 - JavaScript est un langage dynamique ;
 - JavaScript est un langage principalement côté client ;
 - JavaScript est un langage interprété ;
 - JavaScript est un langage orienté objet.
 - **Standardisation:** En 1997, JavaScript est normalisé par l'ECMA International en tant que ECMAScript (ES).
- > ES3 (1999) : Ajout de méthodes pour la manipulation de chaîne de caractères, tables, et objets.
- > ES5 (2009) : Support de JSON, ajout de plus de méthode de table, et autres améliorations.
- > ES6 (2015) : Révolution majeure avec des fonctionnalités dont les classes, les modules, et les promesses.
- > Versions ultérieures (ES7, ES8, etc.) : Ajout de fonctionnalités telles que async/await, static, etc.
- ECMAScript** est une spécification qui décrit les fonctionnalités, la syntaxe et le comportement du langage JavaScript. Les fabricants de navigateurs implémentent ces spécifications à leurs manières, tant qu'ils assurent que le fonctionnement de JavaScript respecte les règles et exigences définies par ECMA.

Langage compilé:



67

Langage interprété:



68

1. Introduction:

Dans le navigateur, l'interpréteur JavaScript (comme V8 pour Chrome ou SpiderMonkey pour Firefox) exécute le code ligne par ligne instantanément, sans compilation préalable.

Moteur JavaScript	Navigateur	Rapidité	Compatibilité
V8	Chrome, Edge	Très rapide	Excellente
SpiderMonkey	Firefox	Rapide	Très bonne
JavaScriptCore (Nitro)	Safari	Rapide	Très bonne
Chakra	Ancien Edge (internet explorer)	Moins rapide	Bonne

69

2. Syntaxe et Structure

2.1. Présentation des variables JavaScript : Déclaration

Une variable est un conteneur servant à stocker des informations de manière temporaire, comme une chaîne de caractères (un texte) ou un nombre par exemple. Lorsqu'on stocke une valeur dans une variable, on dit également qu'on assigne une valeur à une variable.

-> Pour déclarer une variable en JavaScript, il faut utiliser le mot clé **var** ou le mot clef **let**.

-> Il faut respecter des règles de nomenclature :

1. Le nom d'une variable doit obligatoirement commencer par une lettre ou un underscore (`_`) et ne doit pas commencer par un chiffre ;
2. Le nom d'une variable ne doit contenir que des lettres, des chiffres et des underscores mais pas de caractères spéciaux ;
3. Le nom d'une variable ne doit pas contenir d'espace.

-> Les noms des variables sont sensibles à la casse en JavaScript. (**texte**, **TEXTE** et **tEXTe**)

-> En JavaScript, certains noms sont réservés et ne peuvent pas être utilisés pour les variables, car ils sont déjà employés pour désigner des éléments du langage.

-> La convention **lower camel** case est généralement utilisée pour nommer les variables en JavaScript, où chaque mot après le premier commence par une majuscule (ex: `monAge`).

70

2. Syntaxe et Structure

Intégration de JavaScript dans HTML:

1. Méthode 1 : Code JavaScript intégré au document html

JavaScript peut être intégré n'importe où dans le document HTML. Il est aussi possible de l'utiliser plusieurs fois dans le même document HTML.

```
<script language="JavaScript">

/* ou // code js */

</script>
```

2. Méthode 2 : Code JavaScript externe

```
<script language="javascript" src="monScript.js"> </script>
```

71

2. Syntaxe et Structure

Les commentaires:

– Pour mettre en commentaires toute une ligne, on utilise le double slash:

// Tous les caractères derrière le // sont ignorés

– Pour mettre en commentaire une partie du texte (éventuellement sur plusieurs lignes) on utilise le /* et le */:

/* Toutes les lignes comprises entre ces repères sont ignorées par l'interpréteur de code */

72

2. Syntaxe et Structure

2.2. Présentation des variables JavaScript : Initialisation

Initialiser une variable consiste à lui attribuer une valeur pour la première fois, soit au moment de sa déclaration, soit après.

- Il est plus efficace d'initialiser une variable lors de sa déclaration, en utilisant l'opérateur = qui sert à assigner une valeur.
- Le propre d'une variable et l'intérêt principal de celles-ci est de pouvoir stocker différentes valeurs.

```
1 // On déclare une variable et initialise la variable
2 let prenom = "Ali";
3
4
5 //On déclare une variable puis on l'initialise ensuite
6 let monAge;
7 monAge = 11;
8
```

```
1 // On déclare une variable et initialise la variable en m
2 let prenom = "Ali";
3
4
5 /* On modifie la valeur stockée dans prénom
6  * Notre variable stocke désormais la valeur "Mathilde"
7  */
8 prenom = "Mathilde";
9
```

73

2. Syntaxe et Structure

2.2. Présentation des variables JavaScript : Initialisation

La différence entre les mots clefs **let** et **var**:

- Lors de la création de JavaScript, et pendant de nombreuses années, le seul mot-clé disponible pour déclarer des variables était **var**.
- Les créateurs de JavaScript ont introduit le mot-clé **let** pour remplacer **var**, jugé source de confusion. Ils en ont également profité pour résoudre certains problèmes associés à **var**.

Il existe trois principales différences entre les variables déclarées avec **var** et celles déclarées avec **let**, que nous allons illustrer :

1. La remontée ou « **hoisting** » des variables
2. La redéclaration de variables
3. La portée des variables

74

2. Syntaxe et Structure

2.2. Présentation des variables JavaScript : Initialisation

1. La remontée ou « hoisting » des variables

Le "hoisting" (remontée) permettait aux variables déclarées avec `var` d'être utilisées avant leur déclaration dans le code, car JavaScript traitait ces déclarations en premier. Ce comportement, jugé inadapté, a été corrigé avec `let`, où les variables doivent être déclarées avant d'être utilisées. Cette modification vise à encourager des scripts plus structurés et clairs.

```
1 //Ceci fonctionne
2 prenom = "Ali";
3 var prenom;
4
5
6 //Ceci ne fonctionne pas et renvoie une erreur
7 monAge = 22;
8 let monAge;
9
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[Running] node "d:\cours-javascript\cours.js"
d:\cours-javascript\cours.js:7
monAge = 22;
    ^
ReferenceError: Cannot access 'monAge' before initialization
    at Object.<anonymous> (d:\cours-javascript\cours.js:7:8)
    at Module._compile (internal/modules/cjs/loader.js:1085:14)
    at Object.Module._extensions..js (internal/modules/cjs/loader.js:1114:10)
    at Module.load (internal/modules/cjs/loader.js:950:32)
    at Function.Module._load (internal/modules/cjs/loader.js:790:12)
    at Function.executeUserEntryPoint [as runMain] (internal/modules/run_main.js:75:12)
    at internal/main/run_main_module.js:17:47
[Done] exited with code=1 in 0.281 seconds
```

2. Syntaxe et Structure

2.2. Présentation des variables JavaScript : Initialisation

2. La redéclaration de variables

Avec `var`, il était possible de redéclarer plusieurs fois la même variable, ce qui modifiait sa valeur. Avec `let`, cela n'est plus autorisé : il suffit d'affecter une nouvelle valeur à la variable sans la redéclarer. Ce changement a été fait pour améliorer la clarté et la pertinence du code, évitant ainsi des redéfinitions inutiles.

```
1 //Ceci fonctionne
2 var prenom = "Ali";
3 var prenom = "Mathilde";
4
5
6 //Ceci ne fonctionne pas et renvoie une erreur
7 let monAge = 22;
8 let monAge = 14;
9
```

```
[Running] node "d:\cours-javascript\cours.js"
d:\cours-javascript\cours.js:8
let monAge = 14;
    ^
SyntaxError: Identifier 'monAge' has already been declared
    at wrapSafe (internal/modules/cjs/loader.js:1001:16)
    at Module._compile (internal/modules/cjs/loader.js:1049:27)
    at Object.Module._extensions..js (internal/modules/cjs/loader.js:1114:10)
    at Module.load (internal/modules/cjs/loader.js:950:32)
    at Function.Module._load (internal/modules/cjs/loader.js:790:12)
    at Function.executeUserEntryPoint [as runMain] (internal/modules/run_main.js:75:12)
    at internal/main/run_main_module.js:17:47
[Done] exited with code=1 in 0.233 seconds
```

2. Syntaxe et Structure

2.2. Présentation des variables JavaScript : Initialisation

3. La portée des variables

La « portée » d'une variable désigne l'endroit où cette variable va pouvoir être utilisée dans un script. Il est un peu tôt pour vous expliquer ce concept puisque pour bien le comprendre il faut déjà savoir ce qu'est une fonction.

Vous pouvez pour le moment retenir si vous le souhaitez que les variables déclarées avec `var` et celles avec `let` au sein d'une fonction ne vont pas avoir la même portée, c'est-à-dire qu'on ne va pas pouvoir les utiliser aux mêmes endroits.

- `var` : utilisé pour déclarer une variable globale (function scope) ;
- `let` : utilisé pour déclarer une variable dont la portée est limitée à un bloc (block scope) ;
- `const` : permet de déclarer une variable qui doit avoir une valeur initiale et ne peut pas être réassignée.

Le choix de la déclaration des variables : plutôt avec `let` ou plutôt avec `var`?

La déclaration des variables avec `let` correspond à la nouvelle syntaxe, tandis que `var`, l'ancienne, est destinée à disparaître. Il est donc **recommandé d'utiliser systématiquement `let`** pour déclarer vos variables, et c'est ce que nous utiliserons dans ce cours.

77

2. Syntaxe et Structure

2.3. Les types de données en JavaScript

-> Les variables en JavaScript peuvent stocker différents types de valeurs, comme du texte ou des nombres.

-> Il n'est pas nécessaire de préciser à l'avance le type de valeur qu'une variable va stocker, contrairement à d'autres langages de programmation.

-> JavaScript détecte automatiquement le type de la valeur stockée dans une variable.

-> On peut ensuite effectuer des opérations adaptées au type de la variable, ce qui est très pratique.

-> Une même variable peut contenir des valeurs de différents types au fil du temps, sans se préoccuper de la compatibilité.

-> Par exemple, une variable peut stocker une chaîne de caractères à un moment donné, puis un nombre à un autre moment.

```
D: > cours-javascript > JS cours.js > ...
1 //Ceci fonctionne
2 let variable = 25;
3
4 variable = "Ali";
5
6 variable = [1,2,3];
7
```

78

2. Syntaxe et Structure

2.3. Les types de données en JavaScript

En JavaScript, il existe 7 types de valeurs différents. Chaque valeur qu'on va pouvoir créer et manipuler en JavaScript va obligatoirement appartenir à l'un de ces types. Ces types sont les suivants :

1. **String** ou « chaîne de caractères » en français ;
2. **Number** ou « nombre » en français ;
3. **Boolean** ou « booléen » en français ;
4. **Null** ou « nul / vide » en français ;
5. **Undefined** ou « indéfini » en français ;
6. **Symbol** ou « symbole » en français ;
7. **Object** ou « objet » en français ;

Il est crucial de connaître le type des données, car elles sont manipulées différemment selon leur type, ce qui permet de créer des scripts fonctionnels.

79

2. Syntaxe et Structure

2.3. Les types de données en JavaScript

Le type chaîne de caractères ou string:

Le premier type de données qu'une variable va pouvoir stocker est le type String ou chaîne de caractères. Une chaîne de caractères est une séquence de caractères, ou ce qu'on appelle communément un texte. Toute valeur stockée dans une variable en utilisant des guillemets ou des apostrophes sera considérée comme une chaîne de caractères, et ceci même dans le cas où nos caractères sont à priori des chiffres comme "29" par exemple.

```
1 let prenom = "Je m'appelle Pierre";
2 let age = 29;
3 let age2 = '29';
4
5 console.log('Type de prenom : ' + typeof prenom);
6 console.log('Type de age : ' + typeof age);
7 console.log('Type de age2 : ' + typeof age2);
8
```

```
[Running] node "d:\cours-javascript\cours.js"
Type de prenom : string
Type de age : number
Type de age2 : string
[Done] exited with code=0 in 0.179 seconds
```

80

2. Syntaxe et Structure

2.3. Les types de données en JavaScript

Le type nombre ou number

Les variables en JavaScript vont également pouvoir stocker des nombres. En JavaScript, et contrairement à la majorité des langages, il n'existe qu'un type prédéfini qui va regrouper tous les nombres qu'ils soient positifs, négatifs, entiers ou décimaux (à virgule) et qui est le type **Number**.

```
1 let x = 12;
2 let y = 3.14;
3 let z = -2;
4
5
6 console.log("Le type de x est : " + typeof x);
7 console.log("Le type de y est : " + typeof y);
8 console.log("Le type de z est : " + typeof z);
9
```

```
[Running] node "d:\cours-javascript\cours.js"
Le type de x est : number
Le type de y est : number
Le type de z est : number

[Done] exited with code=0 in 0.168 seconds
```

81

2. Syntaxe et Structure

2.3. Les types de données en JavaScript

Le type nombre ou number

Les variables en JavaScript vont également pouvoir stocker des nombres. En JavaScript, et contrairement à la majorité des langages, il n'existe qu'un type prédéfini qui va regrouper tous les nombres qu'ils soient positifs, négatifs, entiers ou décimaux (à virgule) et qui est le type **Number**.

```
1 let x = 12;
2 let y = 3.14;
3 let z = -2;
4
5
6 console.log("Le type de x est : " + typeof x);
7 console.log("Le type de y est : " + typeof y);
8 console.log("Le type de z est : " + typeof z);
9
```

```
[Running] node "d:\cours-javascript\cours.js"
Le type de x est : number
Le type de y est : number
Le type de z est : number

[Done] exited with code=0 in 0.168 seconds
```

82

2. Syntaxe et Structure

2.3. Les types de données en JavaScript

Le type de données booléen (boolean)

Une variable en JavaScript peut encore stocker une valeur de type Boolean, c'est-à-dire un booléen.

Un booléen, en algèbre, est une valeur binaire (soit 0, soit 1). En informatique, le type booléen est un type qui ne contient que deux valeurs : les valeurs **true** (vrai) et **false** (faux).

```
1  let vrai = true;
2  let faux = false;
3
4  let resultat = 8>4;
5
6  console.log("Le type de vrai est : " + typeof vrai);
7  console.log("Le type de faux est : " + typeof faux);
8  console.log("La valeur affecté à résultat est : " + resultat);
9
```

```
[Running] node "d:\cours-javascript\cours.js"
```

```
Le type de vrai est : boolean
```

```
Le type de faux est : boolean
```

```
La valeur affecté à résultat est : true
```

```
[Done] exited with code=0 in 0.267 seconds
```

83

2. Syntaxe et Structure

2.3. Les types de données en JavaScript

Les types de valeurs null et undefined

Les types **Null** et **Undefined** sont particuliers, car chacun ne contient qu'une seule valeur : null et undefined.

null indique explicitement l'absence de valeur connue et doit être assigné de manière explicite, tandis que **undefined** désigne une variable à laquelle aucune valeur n'a été affectée.

```
1  let nul = null;
2  let ind;
3
4  console.log('Type de nul : ' + typeof nul);
5  console.log('Type de ind : ' + typeof ind);
6
```

```
[Running] node "d:\cours-javascript\cours.js"
```

```
Type de nul : object
```

```
Type de ind : undefined
```

```
[Done] exited with code=0 in 0.177 seconds
```

84

2. Syntaxe et Structure

2.4. Les opérateurs en Javascript

Qu'est-ce qu'un opérateur ?

Un opérateur est un symbole qui va être utilisé pour effectuer certaines actions notamment sur les variables et leurs valeurs. Par exemple, l'opérateur `*` va nous permettre de multiplier les valeurs de deux variables, tandis que l'opérateur `=` va nous permettre d'affecter une valeur à une variable.

Il existe différents types d'opérateurs qui vont nous servir à réaliser des opérations de types différents. Les plus fréquemment utilisés sont :

- Les opérateurs arithmétiques: `+`, `-`, `/` ...
- Les opérateurs d'assignation: `=`, `+=` ...
- Les opérateurs de comparaison: `<`, `>`, `==` ...
- Les opérateurs d'incrément et décrémentation: `++`, `--` ...
- Les opérateurs logiques: `and`, `or` ...
- L'opérateur ternaire
- L'opérateur de concaténation

85

2. Syntaxe et Structure

2.4. Les opérateurs en Javascript

Les opérateurs arithmétiques et opérateurs d'incrément et décrémentation:

Les opérateurs arithmétiques vont nous permettre d'effectuer toutes sortes d'opérations mathématiques entre les valeurs de type Number contenues dans différentes variables.

Opérateur	Signification
<code>+</code>	Addition
<code>-</code>	Soustraction
<code>*</code>	Multiplication
<code>**</code>	Puissance
<code>/</code>	Division
<code>%</code>	Modulo (Reste de la division)
<code>++</code>	Incrément
<code>--</code>	Décrément

86

2. Syntaxe et Structure

2.4. Les opérateurs en Javascript

Les opérateurs d'affectation :

Les opérateurs d'affectation vont nous permettre, comme leur nom l'indique, d'affecter une certaine valeur à une variable.

Opérateur	Exemple d'utilisation	Signification	Exemple complet	Résultat
=	x = y	x prend la valeur de y	let x = 5	x vaut 5
+=	x += y	x = x + y	let x = 10; x += 5;	x vaut 15
-=	x -= y	x = x - y	let x = 10; x -= 5;	x vaut 5
*=	x *= y	x = x * y	let x = 10; x *= 5;	x vaut 50
/=	x /= y	x = x / y	let x = 10; x /= 5;	x vaut 2
%=	x %= y	x = x % y	let x = 10; x %= 5;	x vaut 0
**=	x **= y	x = x ** y ⇔ x = x à la puissance y	let x = 3; x **= 2;	x vaut 9

2. Syntaxe et Structure

2.4. Les opérateurs en Javascript

Les opérateurs de comparaison :

Les opérateurs de comparaison nous permettent, comme leur nom l'indique, de comparer deux valeurs afin de déterminer si elles sont égales, différentes ou si l'une est supérieure ou inférieure à l'autre.

Opérateur	Signification	Exemple
==	Egal à (comparaison des valeurs)	let x = 5; let y = "5"; let z=(x==y); //z=true
===	Egal à (comparaison de la valeur et du type)	let x = 5; let y = "5"; let z=(x===y); //z=false
!=	Différent de (n'est pas égal à)	let x = 5; let y = 5; let z=(x!=y); //z=false
!==	Type ou valeur différente	let x = 5; let y = "5"; let z=(x!==y); //z=true
>	Supérieur à	let x = 5; let y = 5; let z=(x>y); //z=false
<	Inférieur à	let x = 5; let y = 5; let z=(x<y); //z=false
>=	Supérieur ou égal à	let x = 5; let y = 5; let z=(x>=y); //z=true
<=	Inférieur ou égal à	let x = 5; let y = 5; let z=(x<=y); //z=true

2. Syntaxe et Structure

2.4. Les opérateurs en Javascript

Utilisation des opérateurs de comparaison ;

```
D: > cours-javascript > JS cours.js > ...
```

```
1 let x = 4; //On stocke le chiffre 4 dans x
2
3 /*Les comparaisons sont effectuées avant l'affectation. Le JavaScript va donc
4 *commencer par comparer et renvoyer true ou false et nous allons stocker ce
5 *résultat dans nos variables test*/
6 let test1 = x == 4;
7 let test2 = x === 4;
8 let test3 = x == '4';
9 let test4 = x === '4';
10 let test5 = x != '4';
11 let test6 = x !== '4';
12 let test7 = x > 4;
13 let test8 = x >= 4;
14 let test9 = x < 4;
15
16 console.log('Valeur dans x égale à 4 (en valeur) ? : ' + test1 +
17 '\nValeur dans x égale à 4 (valeur & type) ? : ' + test2 +
18 '\nValeur dans x égale à "4" (en valeur) ? : ' + test3 +
19 '\nValeur dans x égale à "4" (valeur & type) ? : ' + test4 +
20 '\nValeur dans x différente de "4" (en valeur) ? : ' + test5 +
21 '\nValeur dans x différente de "4" (valeur & type) ? : ' + test6 +
22 '\nValeur dans x strictement supérieure à 4 ? : ' + test7 +
23 '\nValeur dans x supérieure ou égale à 4 ? : ' + test8 +
24 '\nValeur dans x strictement inférieure à 4 ? : ' + test9);
25
```

```
[Running] node "d:\cours-javascript\cours.js"
```

```
Valeur dans x égale à 4 (en valeur) ? : true
Valeur dans x égale à 4 (valeur & type) ? : true
Valeur dans x égale à "4" (en valeur) ? : true
Valeur dans x égale à "4" (valeur & type) ? : false
Valeur dans x différente de "4" (en valeur) ? : false
Valeur dans x différente de "4" (valeur & type) ? : true
Valeur dans x strictement supérieure à 4 ? : false
Valeur dans x supérieure ou égale à 4 ? : true
Valeur dans x strictement inférieure à 4 ? : false
```

```
[Done] exited with code=0 in 0.297 seconds
```

89

2. Syntaxe et Structure

2.4. Les opérateurs en Javascript

Les opérateurs logiques :

Ils sont utilisés pour combiner ou inverser des expressions booléennes, facilitant ainsi la prise de décisions dans le code.

Ils permettent de vérifier si plusieurs conditions sont vraies:

Opérateur	Signification
&&	ET logique
	OU logique
!	NON logique

90

2. Syntaxe et Structure

2.4. Les opérateurs en Javascript

L'opérateur ternaire ;

L'opérateur ternaire est une forme concise de structure conditionnelle qui permet d'évaluer une expression et de retourner une valeur basée sur un test booléen.

Il se compose de trois parties : **une condition**, **une valeur** si la condition est **vraie**, et **une valeur** si la condition est **fausse**. Sa syntaxe est la suivante :

condition ? valeurSiVraie : valeurSiFausse ;

Cela permet d'écrire des conditions simples de manière plus compacte, rendant le code plus lisible.

```
1  const nom = 'Julien'
2
3  // Condition ternaire ou condition if else en une ligne :
4  const color = (nom === 'Julien')? 'blue' : 'green'
5  // Si la variable name est égale à "Julien", la couleur est "blue" sinon, elle est "green"
6
7  console.log(color)
8  // Sortie : "blue"
9
```

91

2. Syntaxe et Structure

2.4. Les opérateurs en Javascript

L'opérateur de concaténation :

Concaténer signifie littéralement « mettre bout à bout ». l'opérateur de concaténation est le signe **+**. Faites bien attention ici : lorsque le signe **+** est utilisé avec deux nombres, il sert à les additionner. Lorsqu'il est utilisé avec autre chose que deux nombres, il sert d'opérateur de concaténation.

```
1  let x = 28 + 1; //Le signe "+" est ici un opérateur arithmétique
2  let y = 'Bonjour';
3  let z = x + ' ans'; //Le signe "+" est ici un opérateur de concaténation
4
5
6  console.log(y + ', je m\'appelle Pierre, j\'ai ' + z);
7
```

[Running] node "d:\cours-javascript\cours.js"
 Bonjour, je m'appelle Pierre, j'ai 29 ans

[Done] exited with code=0 in 0.161 seconds

92

2. Syntaxe et Structure

2.4. Les opérateurs en Javascript

Les constantes:

Une constante est similaire à une variable au sens où c'est également un conteneur pour une valeur. Cependant, à la différence des variables, on ne va pas pouvoir modifier la valeur d'une constante.

```
D: > cours-javascript > JS cours.js > ...
1  const prenom = "Pierre";
2  const age = 27;
3
4  //Ceci déclenche une erreur
5  age = 24;
6

[Running] node "d:\cours-javascript\cours.js"
d:\cours-javascript\cours.js:4
age = 24;
   ^
TypeError: Assignment to constant variable.
    at Object.<anonymous> (d:\cours-javascript\cours.js:4:5)
    at Module._compile (internal/modules/cjs/loader.js:1085:14)
    at Object.Module._extensions..js (internal/modules/cjs/loader.js:1114:10)
    at Module.load (internal/modules/cjs/loader.js:950:32)
    at Function.Module._load (internal/modules/cjs/loader.js:790:12)
    at Function.executeUserEntryPoint [as runMain] (internal/modules/run_main.js:75:12)
    at internal/main/run_main_module.js:17:47

[Done] exited with code=1 in 0.177 seconds
```

Exercice TP 4:

Quel est le résultat de l'exécution de chacune de ces lignes dans la console ? Pourquoi ?

1. var a; typeof a;
2. var s = '1s'; s++;
3. !! "false"
4. !! undefined
5. typeof -Infinity
6. 10 % "0"
7. undefined == null
8. false === ""
9. typeof "2E+2"
10. a = 3e+3; a++;

2. Syntaxe et Structure

2.5. Les tableaux en JavaScript et l'objet global Array

Définition:

Les tableaux sont des éléments qui vont pouvoir contenir plusieurs valeurs. En JavaScript, comme les tableaux sont avant tout des objets, il peut paraître évident qu'un tableau va pouvoir contenir plusieurs valeurs comme n'importe quel objet.

-> Le principe des tableaux est relativement simple : un indice ou clef va être associé à chaque valeur du tableau. Pour récupérer une valeur dans le tableau, on va utiliser les indices qui sont chacun unique dans un tableau.

-> Les tableaux ne sont pas des valeurs primitives. Cependant, nous ne sommes pas obligés d'utiliser le constructeur Array() avec le mot clef new pour créer un tableau en JavaScript.

95

2. Syntaxe et Structure

2.5. Les tableaux en JavaScript et l'objet global Array

Création d'un tableau en JavaScript :

```
JS cours.js > ...
1 // Création d'un tableau contenant des nombres
2 const nombres = [1, 2, 3, 4, 5];
3
4 // Création d'un tableau contenant des chaînes de caractères
5 const fruits = ['Pomme', 'Banane', 'Cerise', 'Orange'];
6
7 // Création d'un tableau contenant un autre tableau
8 const tableau2D = [
9   [1, 2, 3],
10  [4, 5, 6],
11  [7, 8, 9]
12 ];
13
14 // Création d'un tableau contenant des valeurs de types variés
15 const diverseArray = [42, 'Hello', true, null, [1, 2]];
16
```

96

2. Syntaxe et Structure

2.5. Les tableaux en JavaScript et l'objet global Array

Accéder à une valeur dans un tableau:

Lorsqu'on crée un tableau, un indice est automatiquement associé à chaque valeur du tableau. Chaque indice dans un tableau est toujours unique et permet d'identifier et d'accéder à la valeur qui lui est associée. Pour chaque tableau, l'indice 0 est automatiquement associé à la première valeur, l'indice 1 à la deuxième et etc.

```
JS cours.js > ...
1 let prenom = ['Pierre', 'Mathilde', 'Florian', 'Camille'];
2 let ages = [29, 27, 29, 30];
3 let produits = ['Livre', 20, 'Ordinateur', 5, ['Magnets', 100]];
4
5
6
7 // Utilisation de console.log au lieu de innerHTML
8 console.log(prenom[0] + ' possède 1 ' + produits[2]);
9 console.log(prenom[1] + ' a ' + ages[1] + ' ans');
10 console.log(produits[4][1] + ' ' + produits[4][0]);
11
```

```
[Running] node "d:\cours-javascript\cours.js"
Pierre possède 1 Ordinateur
Mathilde a 27 ans
100 Magnets
[Done] exited with code=0 in 0.144 seconds
```

97

2. Syntaxe et Structure

2.5. Les tableaux en JavaScript et l'objet global Array

Les propriétés et les méthodes du Array() :

Le constructeur Array() en JavaScript possède deux propriétés, length pour le nombre d'éléments d'un tableau et prototype, ainsi qu'une trentaine de méthodes utiles, dont certaines sont particulièrement puissantes.

1. **La propriété length** : Elle retourne le nombre d'éléments d'un tableau

```
JS cours.js > ...
1 let ages = [29, 27, 29, 30];
2 console.log(ages.length);
3
```

```
[Running] node "d:\cours-javascript\cours.js"
4
[Done] exited with code=0 in 0.173 seconds
```

98

2. Syntaxe et Structure

2.5. Les tableaux en JavaScript et l'objet global Array

Les propriétés et les méthodes du Array() :

2. La méthode **push()** va nous permettre d'ajouter des éléments en fin de tableau et va retourner la nouvelle taille du tableau

```
JS cours.js > ...
1 let prenom = ['Pierre', 'Mathilde'];
2 let taille = prenom.push('Florian', 'Camille');
3
4 console.log("La nouvelle taille de prenom est : " + taille);
5
```

```
[Running] node "d:\cours-javascript\cours.js"
La nouvelle taille de prenom est : 4
[Done] exited with code=0 in 0.15 seconds
```

99

2. Syntaxe et Structure

2.5. Les tableaux en JavaScript et l'objet global Array

Les propriétés et les méthodes du Array() :

3. La méthode **pop()** va elle nous permettre de supprimer le dernier élément d'un tableau et va retourner l'élément supprimé.

```
JS cours.js > ...
1 let ages = [29, 27, 32];
2 let age = ages.pop();
3
4 console.log("La valeur supprimée est : " + age);
```

```
[Running] node "d:\cours-javascript\cours.js"
La valeur supprimée est : 32
[Done] exited with code=0 in 0.177 seconds
```

100

2. Syntaxe et Structure

2.5. Les tableaux en JavaScript et l'objet global Array

Les propriétés et les méthodes du Array() :

4. La méthode `join()` retourne une chaîne de caractères créée en concaténant les différentes valeurs d'un tableau. Le séparateur utilisé par défaut sera la virgule mais nous allons également pouvoir passer le séparateur de notre choix en argument de `join()`.

```
JS cours.js > ...
1 let ages = [29, 27, 32];
2 let resultat1 = ages.join();
3 let resultat2 = ages.join(' - ');
4
5 console.log("Le premier résultat de join : " + resultat1);
6 console.log("Le deuxième résultat de join : " + resultat2);
```

```
[Running] node "d:\cours-javascript\cours.js"
Le premier résultat de join : 29,27,32
Le deuxième résultat de join : 29 - 27 - 32

[Done] exited with code=0 in 0.155 seconds
```

101

2. Syntaxe et Structure

2.5. Les tableaux en JavaScript et l'objet global Array

Les propriétés et les méthodes du Array() :

5. La méthode `forEach()` : Elle est utilisée pour exécuter une fonction fournie une fois pour chaque élément d'un tableau. Elle est particulièrement utile pour effectuer des opérations sur chaque élément du tableau.

```
JS cours.js > ...
1 const fruits = ['Pomme', 'Banane', 'Cerise', 'Fraise'];
2
3 // Utilisation de forEach pour afficher chaque fruit
4 fruits.forEach(function(fruit, index) {
5     console.log(index + ' : ' + fruit);
6 });
7
```

```
[Running] node "d:\cours-javascript\cours.js"
0: Pomme
1: Banane
2: Cerise
3: Fraise

[Done] exited with code=0 in 0.152 seconds
```

102

2. Syntaxe et Structure

2.5. Les tableaux en JavaScript et l'objet global Array

D'autres méthodes du Array() :

6. shift() : Supprime le premier élément d'un tableau et retourne cet élément.

7. unshift() : Ajoute un ou plusieurs éléments au début d'un tableau et retourne la nouvelle longueur du tableau.

8. slice() : Renvoie une copie superficielle d'une portion d'un tableau dans un nouveau tableau, sans modifier le tableau d'origine.

9. splice() : Change le contenu d'un tableau en supprimant ou en remplaçant des éléments existants et/ou en ajoutant de nouveaux éléments à la place. Elle retourne les éléments supprimés.

10. map() : Crée un nouveau tableau avec les résultats de l'appel d'une fonction fournie sur chaque élément du tableau d'origine.

11. filter() : Crée un nouveau tableau contenant tous les éléments du tableau d'origine qui passent un test fourni par une fonction.

2. Syntaxe et Structure

2.5. Les tableaux en JavaScript et l'objet global Array

D'autres méthodes du Array() :

12. reduce() : Applique une fonction contre un accumulateur et chaque élément du tableau (de gauche à droite) pour réduire le tableau à une seule valeur.

13. find() : Renvoie la première valeur trouvée dans le tableau qui satisfait une fonction de test fournie. Si aucune valeur n'est trouvée, elle retourne undefined.

14. some() : Teste si au moins un élément du tableau passe le test implémenté par la fonction fournie. Elle retourne un booléen.

15. every() : Teste si tous les éléments du tableau passent le test implémenté par la fonction fournie. Elle retourne également un booléen.

Liens utile:

<https://javascript.info/>

<https://developer.mozilla.org/en-US/docs/Web/JavaScript> (MDN)

<https://262.ecma-international.org/14.0/#sec-intro> (ECMA)

<https://www.w3schools.com/js/>

https://www.tutorialspoint.com/javascript/javascript_roadmap.htm

2. Syntaxe et Structure

2.6. Structures de contrôle

On appelle « structure de contrôle » un ensemble d'instructions qui permet de contrôler l'exécution du code. Il existe deux principaux types de structures de contrôle que l'on retrouve dans la plupart des langages de programmation, y compris JavaScript :

1. **Structures de contrôle conditionnelles (ou simplement « conditions »)** : Elles permettent d'exécuter un ensemble d'instructions uniquement si une certaine condition est remplie.
2. **Structures de contrôle de boucles (ou simplement « boucles »)** : Elles permettent d'exécuter un bloc de code de manière répétée tant qu'une condition donnée reste vraie.

2. Syntaxe et Structure

2.6. Structures de contrôle

Les conditions en JavaScript :

Les structures de contrôle conditionnelles (ou plus simplement conditions) vont nous permettre d'exécuter une série d'instructions si une condition donnée est vérifiée ou (éventuellement) une autre série d'instructions si elle ne l'est pas.

En JavaScript, nous disposons des structures conditionnelles suivantes :

1. **La condition if (si)** : Elle permet d'exécuter un bloc de code uniquement si une condition est vraie.
2. **La condition if... else (si... sinon)** : Elle permet d'exécuter un bloc de code si la condition est vraie et un autre bloc si elle est fausse.
3. **La condition if... else if... else (si... sinon si... sinon)** : Elle permet de tester plusieurs conditions successives, en exécutant le bloc de code correspondant à la première condition vraie.
4. **L'instruction switch**

107

2. Syntaxe et Structure

2.6. Structures de contrôle

Les conditions en JavaScript :

-> La condition if

```
D: > cours-javascript > JS cours.js > ...
1  let x = 4;
2  let y = 0;
3
4  if(x > 1){
5    console.log('x contient une valeur strictement supérieure à 1');
6  }
7
8  if(x == y){
9    console.log('x et y contiennent la même valeur');
10 }
11
12 if(y){
13   console.log('La valeur de y est évaluée à true');
14 }
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[Running] node "d:\cours-javascript\cours.js"

x contient une valeur strictement supérieure à 1

[Done] exited with code=0 in 0.155 seconds

108

2. Syntaxe et Structure

2.6. Structures de contrôle

Les conditions en JavaScript :

-> La condition if.....else

```
D: > cours-javascript > JS cours.js > ...
1  let x = 0.5;
2
3  if(x > 1){
4      console.log('x contient une valeur strictement supérieure à 1');
5  }else{
6      console.log('x contient une valeur inférieure ou égale à 1');
7  }
8
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[Running] node "d:\cours-javascript\cours.js"
x contient une valeur inférieure ou égale à 1

[Done] exited with code=0 in 0.173 seconds

109

2. Syntaxe et Structure

2.6. Structures de contrôle

Les conditions en JavaScript :

-> La condition if...else if...else

```
D: > cours-javascript > JS cours.js > ...
1  let x = 0.5;
2
3  if(x > 1){
4      console.log('x contient une valeur strictement supérieure à 1');
5  }else if(x == 1){
6      console.log('x contient la valeur 1');
7  }else{
8      console.log('x contient une valeur strictement inférieure à 1');
9  }
10
```

[Running] node "d:\cours-javascript\cours.js"
x contient une valeur strictement inférieure à 1

[Done] exited with code=0 in 0.159 seconds

110

2. Syntaxe et Structure

2.6. Structures de contrôle

Les conditions en JavaScript :

-> L'instruction switch

L'instruction **switch** va nous permettre d'exécuter un code en fonction de la valeur d'une variable. On va pouvoir gérer autant de situations ou de « cas » que l'on souhaite.

En cela, l'instruction switch représente une alternative à l'utilisation d'un if...else if...else.

```
D: > cours-javascript > JS cours.js > ...
1  let x = 15;
2
3  switch(x){
4      case 2:
5          console.log('x stocke la valeur 2');
6          break;
7      case 5:
8          console.log('x stocke la valeur 5');
9          break;
10     case 10:
11         console.log('x stocke la valeur 10');
12         break;
13     case 15:
14         console.log('x stocke la valeur 15');
15         break;
16     case 20:
17         console.log('x stocke la valeur 20');
18         break;
19     default:
20         console.log('x ne stocke ni 2, ni 5, ni 10, ni 15 ni 20');
21 }
22
```

2. Syntaxe et Structure

2.6. Structures de contrôle

Les boucles en JavaScript :

Les boucles permettent d'exécuter plusieurs fois un bloc de code, c'est-à-dire de faire tourner un code « en boucle » tant qu'une condition donnée est vérifiée.

Cela nous fait gagner un temps précieux lors de l'écriture des scripts. L'utilisation d'une boucle permet de ne rédiger le code à exécuter qu'une seule fois.

En JavaScript, nous avons trois types de boucles :

1. **La boucle while (« tant que »)** : Elle exécute un bloc de code tant qu'une condition est vraie.
2. **La boucle do... while (« faire... tant que »)** : Elle exécute un bloc de code au moins une fois, puis continue tant qu'une condition est vraie.
3. **La boucle for (« pour »)** : Elle permet d'exécuter un bloc de code un nombre précis de fois, en utilisant un compteur.
4. **La boucle for...of** : utilisée pour itérer sur les éléments d'un tableau et tout type itérable

2. Syntaxe et Structure

2.6. Structures de contrôle

Les boucles en JavaScript :

Pour éviter de rester bloqué à l'infini dans une boucle, il faut que la condition donnée soit fausse à un moment donné (pour pouvoir sortir de la boucle).

Les boucles vont donc être essentiellement composées de trois choses :

- **Une valeur** : de départ qui va nous servir à initialiser notre boucle et nous servir de compteur ;
- **Un test ou une condition de sortie** : qui précise le critère de sortie de la boucle ;
- **Un itérateur** : qui va modifier la valeur de départ de la boucle à chaque nouveau passage jusqu'au moment où la condition de sortie est vérifiée. Bien souvent, on incrémentera la valeur de départ.

113

2. Syntaxe et Structure

2.6. Structures de contrôle

Les boucles en JavaScript :

-> La boucle JavaScript do... while

```
D: > cours-javascript > JS cours.js > ...
1 // Initialisation d'une variable pour compter les itérations
2 let compteur = 0;
3
4 // La boucle do... while commence ici
5 do {
6 // Affiche le compteur actuel dans la console
7 console.log("Compteur actuel : " + compteur);
8
9 // Incrémente le compteur de 1
10 compteur++;
11
12 // La condition est vérifiée ici. La boucle continuera tant que le compteur est inférieur à 5
13 } while (compteur < 5);
14
15 // Lorsque le compteur atteint 5, la boucle s'arrête et le programme continue
16 console.log("La boucle est terminée.");
17
```

```
[Running] node "d:\cours-javascript\cours.js"
Compteur actuel : 0
Compteur actuel : 1
Compteur actuel : 2
Compteur actuel : 3
Compteur actuel : 4
La boucle est terminée.

[Done] exited with code=0 in 0.267 seconds
```

114

2. Syntaxe et Structure

2.6. Structures de contrôle

Les boucles en JavaScript :

-> La boucle JavaScript for

```
[Running] node "d:\cours-javascript\cours.js"
```

```
Valeur de i : 0
Valeur de i : 1
Valeur de i : 2
Valeur de i : 3
Valeur de i : 4
La boucle for est terminée.
```

```
[Done] exited with code=0 in 0.176 seconds
```

```
D: > cours-javascript > JS cours.js > ...
```

```
1 // La boucle for commence ici
2 for (let i = 0; i < 5; i++) {
3   // Affiche la valeur actuelle de i dans la console
4   console.log("Valeur de i : " + i);
5 }
6
7 // Après la boucle, on affiche un message indiquant que la boucle est terminée
8 console.log("La boucle for est terminée.");
9
```

115

2. Syntaxe et Structure

2.6. Structures de contrôle

Les boucles en JavaScript :

-> La boucle JavaScript for... of

```
JS cours.js > ...
```

```
1 const couleurs = ['Rouge', 'Vert', 'Bleu', 'Jaune'];
2
3 // Utilisation de for...of pour afficher chaque couleur
4 for (const couleur of couleurs) {
5   console.log(couleur);
6 }
7
```

```
[Running] node "d:\cours-javascript\cours.js"
```

```
Rouge
Vert
Bleu
Jaune
```

```
[Done] exited with code=0 in 0.162 seconds
```

116

2. Syntaxe et Structure

2.6. Structures de contrôle

Les boucles en JavaScript :

-> Intégration avec les instructions continue et break

Continue : Pour sauter une itération de boucle et passer directement à la suivante, on peut utiliser une instruction continue. Cette instruction va nous permettre de sauter l'itération actuelle et de passer directement à l'itération suivante.

```
D: > cours-javascript > JS cours.js > ...
1 // Boucle for qui itère de 0 à 5
2 for (let i = 0; i < 6; i++) {
3     // Si i est égal à 3, on utilise continue pour sauter l'affichage
4     if (i === 3) {
5         continue; // Passe à l'itération suivante de la boucle
6     }
7
8     // Affiche la valeur actuelle de i dans la console
9     console.log("Valeur de i : " + i);
10 }
11
12 // Après la boucle, on affiche un message indiquant que la boucle est terminée
13 console.log("La boucle avec continue est terminée.");
14
```

```
[Running] node "d:\cours-javascript\cours.js"
Valeur de i : 0
Valeur de i : 1
Valeur de i : 2
Valeur de i : 4
Valeur de i : 5
La boucle avec continue est terminée.
```

```
[Done] exited with code=0 in 0.181 seconds
```

117

2. Syntaxe et Structure

2.6. Structures de contrôle

Les boucles en JavaScript :

-> Intégration avec les instructions continue et break

break : utilisée pour quitter une boucle (comme une boucle for, while ou do...while) ou une instruction switch avant qu'elle ne soit terminée. Elle permet de sortir immédiatement de la structure de contrôle en cours, ce qui peut être utile pour interrompre des itérations sous certaines conditions.

```
JS cours.js > ...
1 for (let i = 0; i < 10; i++) {
2     if (i === 5) {
3         break; // Quitte la boucle lorsque i est égal à 5
4     }
5     console.log(i);
6 }
7
```

```
[Running] node "d:\cours-javascript\cours.js"
0
1
2
3
4
```

```
[Done] exited with code=0 in 0.163 seconds
```

118

2. Syntaxe et Structure

2.7. Les fonctions :

-> Les fonctions prédéfinies :

Définition : Une fonction correspond à un bloc de code nommé et réutilisable et dont le but est d'effectuer une tâche précise.

Le langage JavaScript dispose de nombreuses fonctions que nous pouvons utiliser pour effectuer différentes tâches. Les fonctions définies dans le langage sont appelées fonctions prédéfinies ou fonctions prêtes à l'emploi car il nous suffit de les appeler pour nous en servir.

Exemple:

```
D: > cours-javascript > JS cours.js > ...
1 // Générer un nombre aléatoire entre 0 (inclus) et 1 (exclus)
2 const nombreAleatoire = Math.random();
3
4 // Afficher le nombre aléatoire dans la console
5 console.log("Nombre aléatoire généré :", nombreAleatoire);
6
```

```
[Running] node "d:\cours-javascript\cours.js"
Nombre aléatoire généré : 0.23213463978735605

[Done] exited with code=0 in 0.162 seconds
```

Autres exemples :

parseInt() : Convertit une chaîne de caractères en un entier. **alert() :** Affiche une boîte de dialogue d'alerte avec un message. **setTimeout() :** Exécute une fonction après un certain délai.

119

2. Syntaxe et Structure

2.7. Les fonctions :

-> Les fonctions personnalisées :

-> On crée une fonction personnalisée grâce au mot clef **function** ;

-> Si une fonction a besoin qu'on lui passe des valeurs pour fonctionner, alors on définira des paramètres lors de sa définition.

```
D: > cours-javascript > JS cours.js > ...
1 // Déclaration d'une fonction sans argument
2 function maFonction() {
3 // Corps de la fonction (peut rester vide)
4 }
5
```

```
D: > cours-javascript > JS cours.js > ...
1 // Déclaration d'une fonction avec deux arguments
2 function maFonction2(arg1,arg2) {
3
4 }
5
```

-> Pour exécuter le code d'une fonction, il faut l'appeler. Pour cela, il suffit d'écrire son nom suivi d'un couple de parenthèses en passant éventuellement des arguments dans les parenthèses ;

```
D: > cours-javascript > JS cours.js
1 maFonction();
2 maFonction2("Ali",28);
3
```

120

2. Syntaxe et Structure

2.7. Les fonctions :

-> La notion de portée des variables : Définition

Il est essentiel de saisir la notion de « portée » des variables lorsque l'on travaille avec des fonctions en JavaScript.

Définition de la portée : La portée d'une variable fait référence à la zone du script dans laquelle elle est accessible. En effet, toutes les variables ne sont pas disponibles à chaque endroit d'un script, ce qui limite leur utilisation.

Il existe deux espaces de portée principaux :

1. **Espace global :** Il englobe l'ensemble du script, à l'exception du code à l'intérieur des fonctions. Les variables déclarées dans cet espace sont accessibles partout dans le script.
2. **Espace local :** À l'opposé, l'espace local est celui d'une fonction spécifique. Les variables déclarées à l'intérieur d'une fonction ne peuvent être utilisées que dans le cadre de cette fonction.

Cette distinction entre portée globale et locale est cruciale pour éviter les conflits de noms de variables et pour gérer correctement l'accès aux données dans votre code.

121

2. Syntaxe et Structure

2.7. Les fonctions :

-> La notion de portée des variables : Exemple pratique

```
> cours-javascript > JS cours.js > ...
1 // On déclare deux variables globales
2 let x = 5;
3 var y = 10;
4
5 // On définit une première fonction qui utilise les variables globales
6 function portee1() {
7   console.log('Depuis portee1() :');
8   console.log('x = ' + x);
9   console.log('y = ' + y);
10 }
11
12 // On définit une deuxième fonction qui définit des variables locales
13 function portee2() {
14   let a = 1;
15   var b = 2;
16   console.log('Depuis portee2() :');
17   console.log('a = ' + a);
18   console.log('b = ' + b);
19 }
```

Appels:

```
30 // On pense bien à exécuter nos fonctions
31 portee1();
32 portee2();
```

Résultat:

```
[Running] node "d:\cours-javascript\cours.js"
Depuis portee1() :
x = 5
y = 10
Depuis portee2() :
a = 1
b = 2
```

122

2. Syntaxe et Structure

2.7. Les fonctions :

-> La notion de portée des variables : Exemple pratique

Résultat:

```

20
21 // On définit une troisième fonction qui définit également des variables locales
22 function portee3() {
23     let x = 20;
24     var y = 40;
25     console.log('Depuis portee3() :');
26     console.log('x = ' + x);
27     console.log('y = ' + y);
28 }
29

```

```

[Running] node "d:\cours-javascript\cours.js"
Depuis portee1() :
x = 5
y = 10
Depuis portee2() :
a = 1
b = 2
Depuis portee3() :
x = 20
y = 40

[Done] exited with code=0 in 0.175 seconds

```

123

2. Syntaxe et Structure

2.7. Les fonctions :

-> Les valeurs de retour des fonctions

Une valeur de retour est le résultat qu'une fonction renvoie à l'endroit où elle a été appelée. Cela permet d'utiliser le résultat de cette fonction dans d'autres parties du code.

Pour qu'une fonction retourne une valeur, on utilise l'instruction `return`, suivie de la valeur ou de l'expression à renvoyer.

```

D: > cours-javascript > JS cours.js > ...
1 function addition(a, b) {
2     return a + b; // La fonction retourne la somme de a et b
3 }
4
5 let resultat = addition(3, 4); // resultat vaudra 7
6

```

Il est important de noter qu'il n'est pas nécessaire qu'une fonction renvoie une valeur. Si aucune instruction `return` n'est utilisée, la fonction renverra `undefined` par défaut.

Lorsqu'on utilise des fonctions prédéfinies en JavaScript, il est essentiel de vérifier leur valeur de retour, car cela peut affecter le comportement de votre code.

124

2. Syntaxe et Structure

Exercice TP 5:

Gestion de panier de fruits

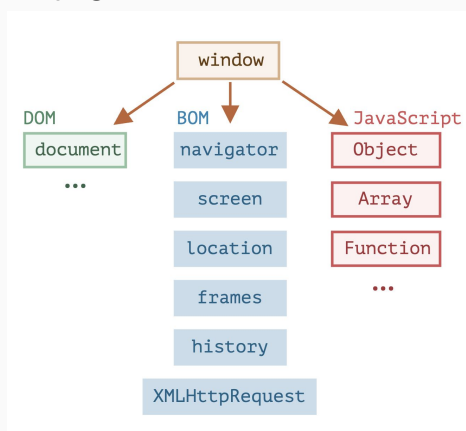
1. Crée un tableau vide **panier**.
2. Crée une fonction **ajouterFruit(fruit)** pour ajouter un fruit.
3. Crée une fonction **retirerFruit(fruit)** qui retire une occurrence du fruit.
4. Crée une fonction **afficherPanier()** pour voir les fruits du panier.
5. Teste ton code en ajoutant « pomme, banane, pomme, orange », retire une "pomme", puis affiche le panier.
6. Crée une fonction **compterFruit(fruit)** qui compte le nombre de fois qu'un fruit est présent.

125

3. Manipulation du DOM

3.1. Introduction au Browser Object Model (BOM)

Le BOM (Browser Object Model) est un ensemble d'objets fournis par le navigateur qui permet aux développeurs d'interagir avec les éléments de la fenêtre du navigateur, comme l'historique, les URL, et les fenêtres, en plus du contenu HTML de la page.

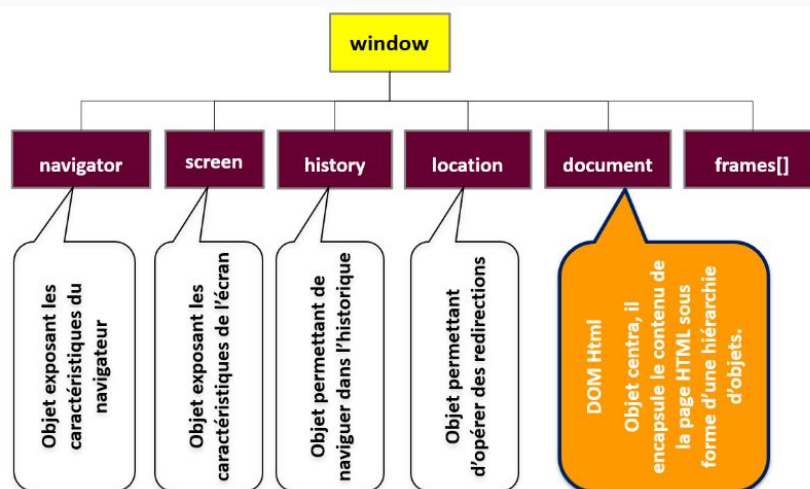


126

3. Manipulation du DOM

3.1. Introduction au Browser Object Model (BOM)

Le BOM (Browser Object Model) est un ensemble d'objets fournis par le navigateur qui permet aux développeurs d'interagir avec les éléments de la fenêtre du navigateur, comme l'historique, les URL, et les fenêtres, en plus du contenu HTML de la page.

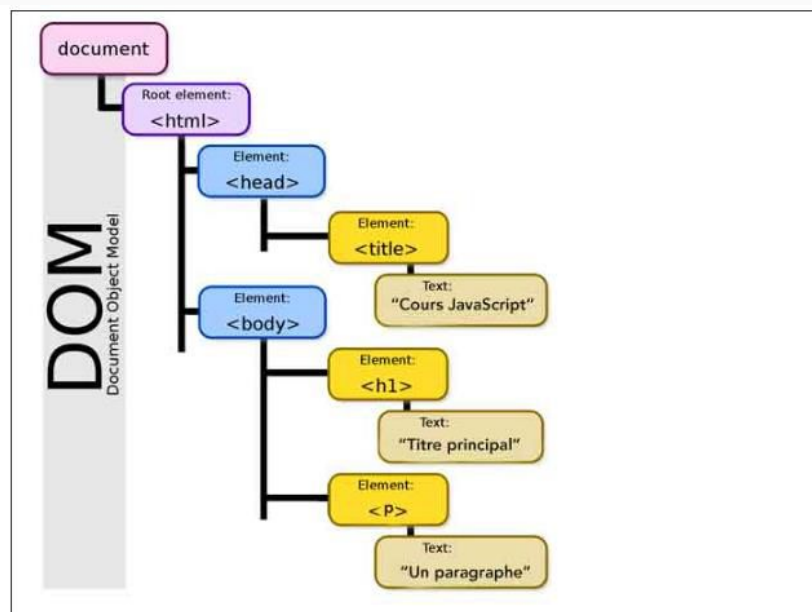


127

3. Manipulation du DOM

3.2. DOM ou Document Object Model

```
<> html.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Cours JavaScript</title>
5      <meta charset="utf-8">
6    </head>
7
8    <body>
9      <h1>Titre principal</h1>
10     <p>Un premier paragraphe</p>
11   </body>
12 </html>
13
14
```



128

3. Manipulation du DOM

3.2. DOM ou Document Object Model

Le DOM est une interface de programmation pour des documents HTML qui représente le document (la page web actuelle) sous une forme qui permet aux langages de script comme le JavaScript d'y accéder et d'en manipuler le contenu et les styles.

Lorsque le navigateur est chargé d'afficher une page Web, il génère automatiquement un modèle objet du **document**. Ce modèle objet est une représentation en forme d'arborescence de la page, constituée d'objets de type **Node** (noeuds).

Les navigateurs exploitent cette arborescence pour faciliter la manipulation des éléments, notamment pour appliquer des styles aux bons éléments. De plus, il est possible d'interagir avec ce modèle objet en utilisant un langage de script tel que JavaScript.

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Les objets Node ou noeuds du DOM

Le terme « noeud » est un terme générique qui sert à désigner tous les objets contenus dans le DOM. A l'extrémité de chaque branche du DOM se trouve un noeud.

A partir du noeud racine qui est le noeud HTML on voit que 3 branches se forment : une première qui va aboutir au noeud HEAD, une deuxième qui aboutit à un noeud #text et une troisième qui aboutit à un noeud BODY.

De nouvelles branches se créent ensuite à partir des noeuds HEAD et BODY et etc.

<https://fr.javascript.info/dom-nodes>

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Pourquoi accéder au DOM ?

- Pour faire des opérations usuelles :
- Récupérer la référence d'un élément(et modifier son contenu.)
- Récupérer la référence d'un ensemble d'éléments
- Modifier les attributs d'un élément
- Modifier les styles d'un élément
- Ajouter/supprimer des classes CSS à un élément
- Ajouter un ou plusieurs éléments au DOM
-

131

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Accéder aux éléments dans un document avec JavaScript et modifier leur contenu

1. Accéder à un élément en fonction de son identité

La méthode **getElementsByTagName()** permet de sélectionner des éléments en fonction de leur nom et renvoie un objet **HTMLCollection** qui consiste en une liste d'éléments correspondant au nom de balise passé en argument.

A noter que cette liste est mise à jour en direct (ce qui signifie qu'elle sera modifiée dès que l'arborescence DOM le sera).

Lorsqu'on utilise **getElementsByTagName()** avec un objet Document, la recherche se fait dans tout le **document** tandis que lorsqu'on utilise **getElementsByTagName()** avec un objet **Element**, la recherche se fera dans l'élément en question seulement.

132

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Accéder aux éléments dans un document avec JavaScript et modifier leur contenu

1. Accéder à un élément en fonction de son identité

Exemple :

```

1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Cours JavaScript</title>
5      <meta charset="utf-8">
6      <link rel="stylesheet" href="cours.css">
7      <script src="cours.js" async></script>
8    </head>
9
10   <body>
11     <h1 class="bleu">Titre principal</h1>
12     <p id="p1">Un paragraphe</p>
13     <div>
14       <p>Un paragraphe dans le div</p>
15       <p class="bleu">Un autre paragraphe dans le div</p>
16     </div>
17     <p>Un autre paragraphe</p>
18   </body>
19 </html>
20

```

```

1  //Sélectionne tous les éléments p du document
2  let paras = document.getElementsByTagName('p');
3
4  // "paras" est un objet de HTMLCollection qu'on va manipuler comme un tableau
5  for (valeur of paras) {
6    valeur.style.color = 'blue';
7  }
8

```

Titre principal

Un paragraphe

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

133

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Accéder aux éléments dans un document avec JavaScript et modifier leur contenu

2. Accéder à un élément en fonction de la valeur de son attribut class

Les interfaces **Element** et **Document** vont toutes deux définir une méthode **getElementsByClassName()** qui va renvoyer une liste des éléments possédant un attribut class avec la valeur spécifiée en argument. La liste renvoyée est un objet de l'interface **HTMLCollection** qu'on va pouvoir traiter quasiment comme un tableau.

134

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Accéder aux éléments dans un document avec JavaScript et modifier leur contenu

2. Accéder à un élément en fonction de la valeur de son attribut class

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours JavaScript</title>
5     <meta charset="utf-8">
6     <link rel="stylesheet" href="cours.css">
7     <script src="cours.js" async</script>
8   </head>
9
10  <body>
11    <h1 class="bleu">Titre principal</h1>
12    <p id="p1">Un paragraphe</p>
13    <div>
14      <p>Un paragraphe dans le div</p>
15      <p class="bleu">Un autre paragraphe dans le div</p>
16    </div>
17    <p>Un autre paragraphe</p>
18  </body>
19 </html>
20

```

```

JS cours.js > ...
1 //Sélectionne les éléments avec une class = 'bleu'
2 let bleu = document.getElementsByClassName('bleu');
3
4 // "bleu" est un objet de HTMLCollection qu'on va manipuler comme un tableau
5 for(valeur of bleu){
6   valeur.style.color = 'red';
7 }
8

```

Titre principal

Un paragraphe

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

135

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Accéder aux éléments dans un document avec JavaScript et modifier leur contenu

3. Accéder à un élément en fonction de la valeur de son attribut id

getElementById() est une méthode en JavaScript qui permet de sélectionner un élément HTML d'une page web en utilisant son identifiant (id).

C'est un moyen simple d'accéder à un élément en particulier (si celui-ci possède un id) puisque les id sont uniques dans un document.

```

JS cours.js
1 //Sélectionne l'élément avec un id = 'p1' et modifie la couleur du texte
2 document.getElementById('p1').style.color = 'green';
3

```

Titre principal

Un paragraphe

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

136

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> **Accéder aux éléments dans un document avec JavaScript et modifier leur contenu**

4. Accéder à un élément en fonction de son attribut name

getElementsByName() est une méthode en JavaScript qui permet de récupérer tous les éléments HTML qui ont un attribut name spécifique.

Elle renvoie une collection d'éléments (une **NodeList**) , généralement utilisée dans des formulaires où plusieurs champs partagent le même nom.

137

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> **Accéder aux éléments dans un document avec JavaScript et modifier leur contenu**

5. Accéder à un élément à partir de son sélecteur CSS associé

-> La façon la plus simple d'accéder à un élément dans un document va être de la faire en le ciblant avec le sélecteur CSS qui lui est associé.

-> Deux méthodes nous permettent de faire cela : les méthodes **querySelector()** et **querySelectorAll()**.

-> La méthode **querySelector()** retourne un objet **Element** représentant le **premier** élément dans le document correspondant au sélecteur (ou au groupe de sélecteurs) CSS passé en argument ou la valeur **null** si aucun élément correspondant n'est trouvé.

-> La méthode **querySelectorAll()** renvoie un objet appartenant à l'interface **NodeList**. Les objets **NodeList** sont des collections (des listes) de noeuds.

138

3. Manipulation du DOM

3.2. DOM ou Document Object Model

5. Accéder à un élément à partir de son sélecteur CSS associé

Exemples : (sélection par id)

```

<> html.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Cours JavaScript</title>
5      <meta charset="utf-8">
6      <link rel="stylesheet" href="cours.css">
7      <script src='cours.js' async></script>
8    </head>
9
10   <body>
11     <h1 class='bleu'>Titre principal</h1>
12     <p id='p1'>Un paragraphe</p>
13     <div>
14       <p>Un paragraphe dans le div</p>
15       <p class='bleu'>Un autre paragraphe dans le div</p>
16     </div>
17     <p>Un autre paragraphe</p>
18   </body>
19 </html>
20

```

```

JS cours.js > ...
1  // Sélectionne le paragraphe avec l'id 'p1' et change sa couleur
2  const paragraphe1 = document.querySelector('#p1');
3  paragraphe1.style.color = 'red'; // Change la couleur du texte en rouge
4

```

Titre principal

Un paragraphe

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

139

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Accéder aux éléments dans un document avec JavaScript et modifier leur contenu

5. Accéder à un élément à partir de son sélecteur CSS associé

Exemples : (sélection par class) :

```

<> html.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Cours JavaScript</title>
5      <meta charset="utf-8">
6      <link rel="stylesheet" href="cours.css">
7      <script src='cours.js' async></script>
8    </head>
9
10   <body>
11     <h1 class='bleu'>Titre principal</h1>
12     <p id='p1'>Un paragraphe</p>
13     <div>
14       <p>Un paragraphe dans le div</p>
15       <p class='bleu'>Un autre paragraphe dans le div</p>
16     </div>
17     <p>Un autre paragraphe</p>
18   </body>
19 </html>
20

```

```

JS cours.js > ...
1  // Sélectionne le premier élément avec la classe 'bleu' et change sa taille de police
2  const titreBleu = document.querySelector('.bleu');
3  titreBleu.style.fontSize = '30px'; // Change la taille de la police à 30px
4

```

Titre principal

Un paragraphe

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

140

3. Manipulation du DOM

3.2. DOM ou Document Object Model

5. Accéder à un élément à partir de son sélecteur CSS associé

Exemples : (sélection par tagname) :

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours JavaScript</title>
5     <meta charset="utf-8">
6     <link rel="stylesheet" href="cours.css">
7     <script src="cours.js" async</script>
8   </head>
9
10  <body>
11    <h1 class='bleu'>Titre principal</h1>
12    <p id='p1'>Un paragraphe</p>
13    <div>
14      <p>Un paragraphe dans le div</p>
15      <p class='bleu'>Un autre paragraphe dans le div</p>
16    </div>
17    <p>Un autre paragraphe</p>
18  </body>
19 </html>
20

```

```

1 // Sélectionne le premier paragraphe et change l'arrière-plan
2 const premierParagraphe = document.querySelector('p');
3 premierParagraphe.style.backgroundColor = 'yellow'; // Change l'arrière-plan en jaune
4

```

Titre principal

Un paragraphe

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

141

3. Manipulation du DOM

3.2. DOM ou Document Object Model

5. Accéder à un élément à partir de son sélecteur CSS associé

Exemples : (sélection par combinaison de plusieurs critères CSS) :

```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Cours JavaScript</title>
5     <meta charset="utf-8">
6     <link rel="stylesheet" href="cours.css">
7     <script src="cours.js" async</script>
8   </head>
9
10  <body>
11    <h1 class='bleu'>Titre principal</h1>
12    <p id='p1'>Un paragraphe</p>
13    <div>
14      <p>Un paragraphe dans le div</p>
15      <p class='bleu'>Un autre paragraphe dans le div</p>
16    </div>
17    <p>Un autre paragraphe</p>
18  </body>
19 </html>
20

```

```

1 // Sélectionne le paragraphe avec la classe 'bleu' à l'intérieur d'un div
2 const paragrapheBleuDansDiv = document.querySelector('div p.bleu');
3 paragrapheBleuDansDiv.style.fontWeight = 'bold'; // Change le texte en gras
4 paragrapheBleuDansDiv.style.backgroundColor = 'yellow'; //change la couleur de background
5

```

Titre principal

Un paragraphe

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

142

3. Manipulation du DOM

3.2. DOM ou Document Object Model

5. Accéder à un élément à partir de son sélecteur CSS associé

Exemples : (querySelectorAll())

```

<> html.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Cours JavaScript</title>
5      <meta charset="utf-8">
6      <link rel="stylesheet" href="cours.css">
7      <script src='cours.js' async</script>
8    </head>
9
10   <body>
11     <h1 class='bleu'>Titre principal</h1>
12     <p id='p1'>Un paragraphe</p>
13     <div>
14       <p>Un paragraphe dans le div</p>
15       <p class='bleu'>Un autre paragraphe dans le div</p>
16     </div>
17     <p>Un autre paragraphe</p>
18   </body>
19 </html>
20

```

```

JS cours.js > ...
1  // Sélectionne tous les paragraphes <p> de la page
2  const paragraphes = document.querySelectorAll('p');
3
4  // Parcourt chaque paragraphe et change la couleur du texte en vert
5  paragraphes.forEach(paragraphe => {
6    paragraphe.style.color = 'green';
7  });
8

```

Titre principal

Un paragraphe

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

143

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Accéder aux éléments dans un document avec JavaScript et modifier leur contenu

6. Modifier le contenu d'un noeud :

Jusqu'à présent, nous avons vu différents moyens d'accéder à un élément en particulier dans un document en utilisant le -> DOM.

-> Accéder à un noeud en particulier va nous permettre d'effectuer différentes manipulations sur celui-ci, et notamment de récupérer le contenu de cet élément ou de le modifier.

144

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Accéder aux éléments dans un document avec JavaScript et modifier leur contenu

6. Modifier le contenu d'un noeud :

Pour récupérer le contenu d'un élément ou le modifier, nous allons pouvoir utiliser l'une des propriétés suivantes :

La propriété **innerHTML** de l'interface **Element** permet de récupérer ou de redéfinir la syntaxe HTML interne à un élément ;

La propriété **outerHTML** de l'interface **Element** permet de récupérer ou de redéfinir l'ensemble de la syntaxe HTML interne d'un élément et de l'élément en soi ;

La propriété **textContent** de l'interface **Node** représente le contenu textuel d'un noeud et de ses descendants. On utilisera cette propriété à partir d'un objet **Element** ;

La propriété **innerText** de l'interface **Node** représente le contenu textuel visible sur le document final d'un noeud et de ses descendants. On utilisera cette propriété à partir d'un objet **Element**.

145

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Accéder aux éléments dans un document avec JavaScript et modifier leur contenu

6. Modifier le contenu d'un noeud :

```
<> html.html > ...
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Cours JavaScript</title>
5      <meta charset="utf-8">
6      <link rel="stylesheet" href="cours.css">
7      <script src="cours.js" async</script>
8    </head>
9
10   <body>
11     <h1>Titre principal</h1>
12     <p id="p1">Un paragraphe</p>
13     <div>
14       <p>Un paragraphe dans le div</p>
15       <p id="texte">Un autre paragraphe dans le div</p>
16     </div>
17     <p id="p2">Un autre paragraphe
18       <span style="visibility: hidden">avec du contenu caché</span>
19     </p>
20     <p id="p3"></p>
21   </body>
22 </html>
```

Titre principal

Un paragraphe

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

146

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Accéder aux éléments dans un document avec JavaScript et modifier leur contenu

6. Modifier le contenu d'un noeud :

```
JS cours.js > ...
1 //Accède au contenu HTML interne du div et le modifie
2 document.querySelector('div').innerHTML +=
3   '<ul><li>Elément n°1</li><li>Elément n°2</li></ul>';
4
5 //Accède au HTML du 1er paragraphe du document et le modifie
6 document.querySelector('p').innerHTML = '<h2>Je suis un titre h2</h2>';
7
8 /*Accède au contenu textuel de l'élément avec un id='texte' et le modifie.
9  *Les balises HTML vont ici être considérées comme du texte*/
10 document.getElementById('texte').textContent = '<span>Texte modifié</span>';
11
12 //Accède au texte visible de l'élément avec l'id = 'p2'
13 let texteVisible = document.getElementById('p2').innerText;
14 //Accède au texte (visible ou non) de l'élément avec l'id = 'p2'
15 let texteEntier = document.getElementById('p2').textContent;
16
17 //Affiche les résultats du dessus dans l'élément avec l'id = 'p3'
18 document.getElementById('p3').innerHTML =
19   'Texte visible : ' + texteVisible + '<br>Texte complet : ' + texteEntier;
```

Titre principal

Je suis un titre h2

Un paragraphe dans le div

Texte modifié

- Élément n°1
- Élément n°2

Un autre paragraphe

Texte visible : Un autre paragraphe

Texte complet : Un autre paragraphe avec du contenu caché

147

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Naviguer ou se déplacer dans le DOM en JavaScript grâce aux noeuds

1. Accéder au parent ou à la liste des enfants d'un noeud

-> La propriété **parentNode** de l'interface **Node** renvoie le parent du noeud spécifié dans l'arborescence du DOM ou **null** si le noeud ne possède pas de parent.

-> Pour n'accéder au parent que dans le cas où celui-ci est un noeud **Element**, on utilisera plutôt la propriété **parentElement** de Node qui ne renvoie le parent d'un noeud que s'il s'agit d'un noeud **Element** ou null sinon.

-> La propriété **childNodes** de cette même interface renvoie une liste des noeuds enfants de l'élément donné.

-> A noter que la propriété **childNodes** renvoie tous les noeuds enfants et cela quels que soient leurs types : noeuds élément, noeuds texte, noeuds commentaire, etc.

-> Si on ne souhaite récupérer que les noeuds enfants éléments, alors on utilisera plutôt la propriété **children**

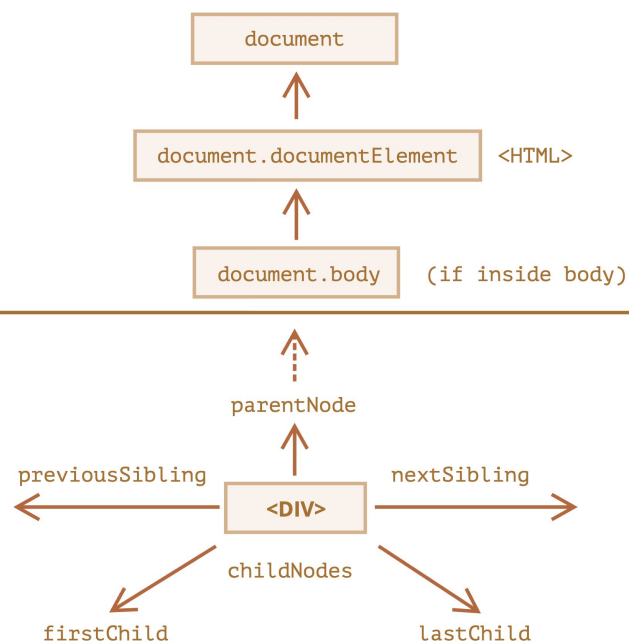
148

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Naviguer ou se déplacer dans le DOM en JavaScript grâce aux noeuds

1. Accéder au parent ou à la liste des enfants d'un noeud



149

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Naviguer ou se déplacer dans le DOM en JavaScript grâce aux noeuds

1. Accéder au parent ou à la liste des enfants d'un noeud

```

<> html.html > ...
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <title>Cours JavaScript</title>
5          <meta charset="utf-8">
6          <link rel="stylesheet" href="cours.css">
7          <script src='cours.js' async></script>
8      </head>
9
10     <body>
11         <h1>Titre principal</h1>
12         <p id='p1'>Un paragraphe <span>avec un span</span></p>
13         <div>
14             <p id='p2'>Un paragraphe dans le div</p>
15             <p>Un autre paragraphe dans le div</p>
16         </div>
17         <p>Un autre paragraphe</p>
18     </body>
19 </html>
20

```

Titre principal

Un paragraphe avec un span

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

150

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Naviguer ou se déplacer dans le DOM en JavaScript grâce aux noeuds

1. Accéder au parent ou à la liste des enfants d'un noeud

```
JS cours.js > ...
1 let p1 = document.getElementById('p1');
2 let p2 = document.getElementById('p2');
3
4 p2.parentNode.style.backgroundColor = 'RGBa(240,160,000,0.5)'; //Orange
5
6 //On accède à tous les noeuds enfants de p1. childNodes renvoie une NodeList
7 let enfantsP1 = p1.childNodes;
8
9 /*On peut ensuite utiliser une boucle forEach() pour tous les manipuler ou
10 *un indice comme pour les tableaux pour manipuler un noeud enfant en
11 *particulier (Le premier enfant a l'indice 0, le deuxième l'indice 1, etc.)*/
12 enfantsP1[1].style.fontWeight = 'bold';
13
14 /*On accède aux noeuds enfants éléments seulement de p1.
15 *children renvoie une HTMLCollection*/
16 let enfantsEltP1 = p1.children;
17
18 //On peut ensuite accéder aux différents enfants comme on le ferait avec un tableau
19 enfantsEltP1[0].style.textDecoration = 'underline';
20
```

Titre principal

Un paragraphe avec un span

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

151

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Naviguer ou se déplacer dans le DOM en JavaScript grâce aux noeuds

2. Accéder à un noeud enfant en particulier à partir d'un noeud parent

* La propriété **firstChild** de l'interface **Node** renvoie le premier noeud enfant direct d'un certain noeud ou **null** s'il n'en a pas.

* La propriété **lastChild**, au contraire, renvoie le dernier noeud enfant direct d'un certain noeud ou **null** s'il n'en a pas.

-> Pour renvoyer le premier et le dernier noeud enfant de type élément seulement d'un certain noeud, on utilisera plutôt les propriétés **firstElementChild** et **lastElementChild** du mixin **ParentNode**.

152

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Naviguer ou se déplacer dans le DOM en JavaScript grâce aux noeuds

2. Accéder à un noeud enfant en particulier à partir d'un noeud parent

```
JS cours.js > ...
1 //On accède au premier noeud enfant de body
2 let bodyFirstChild = document.body.firstChild;
3
4 //On accède au dernier noeud enfant de body
5 let bodyLastChild = document.body.lastChild;
6
7 //On accède au premier noeud enfant élément de body
8 let bodyFirstChildElement = document.body.firstChildElement;
9
10 //On accède au dernier noeud enfant élément de body
11 let bodyLastElementChild = document.body.lastElementChild;
12
```

153

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Naviguer ou se déplacer dans le DOM en JavaScript grâce aux noeuds

3. Accéder au noeud précédent ou suivant un noeud dans l'architecture DOM

* La propriété `previousSibling` de l'interface `Node` renvoie le noeud précédent un certain noeud dans l'arborescence du DOM (en ne tenant compte que des noeuds de même niveau) ou `null` si le noeud en question est le premier.

* La propriété `nextSibling`, au contraire, renvoie elle le noeud suivant un certain noeud dans l'arborescence du DOM (en ne tenant compte que des noeuds de même niveau) ou `null` si le noeud en question est le dernier.

-> Si on souhaite accéder spécifiquement au noeud élément précédent ou suivant un certain noeud, on utilisera plutôt les propriétés `previousElementSibling` et `nextElementSibling` du mixin `NonDocumentTypeChildNode` (mixin implémenté par `Element`).

154

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Naviguer ou se déplacer dans le DOM en JavaScript grâce aux noeuds

3. Accéder au noeud précédent ou suivant un noeud dans l'architecture DOM

<https://fr.javascript.info/dom-navigation>

```
JS cours.js > ...
1 let p1 = document.getElementById('p1');
2
3 let p1PreviousSibling = p1.previousSibling;
4 let p1NextSibling = p1.nextSibling;
5 let p1PreviousElementSibling = p1.previousElementSibling;
6 let p1NextElementSibling = p1.nextElementSibling;
7
```

155

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Ajouter, modifier ou supprimer des éléments du DOM avec JavaScript

1. Créer de nouveaux noeuds :

Pour créer un nouvel élément HTML en JavaScript, nous pouvons utiliser la méthode `createElement()` de l'interface Document.

Pour insérer du texte dans notre noeud élément, on va pouvoir par exemple utiliser la propriété `textContent`.

```
JS cours.js > ...
1 let newP1 = document.createElement('p');
2 let newP2 = document.createElement('p');
3
4 newP1.textContent = 'Paragraphe créé et inséré grâce au JavaScript';
5
6 let newTexte = document.createTextNode('Texte écrit en JavaScript');
7 newP2.textContent = newTexte;
8
```

156

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Ajouter, modifier ou supprimer des éléments du DOM avec JavaScript

2. Insérer un noeud dans le DOM

Il existe différentes méthodes qui nous permettent d'insérer des noeuds dans d'autres noeuds. La différence entre ces méthodes va souvent consister dans la position où le noeud va être inséré.

les deux méthodes **prepend()** et **append()** vont respectivement nous permettre d'insérer un noeud ou du texte avant le premier enfant d'un certain noeud ou après le dernier enfant de ce noeud.

```
JS cours.js > ...
1 let b = document.body;
2 let newP = document.createElement('p');
3 let newTexte = document.createTextNode('Texte écrit en JavaScript');
4
5 newP.textContent = 'Paragraphe créé et inséré grâce au JavaScript';
6
7 //Ajoute le paragraphe créé comme premier enfant de l'élément body
8 b.prepend(newP);
9
10 //Ajoute le texte créé comme dernier enfant de l'élément body
11 b.append(newTexte);
12
```

Paragraphe créé et inséré grâce au J

Titre principal

Un paragraphe avec un span

Un paragraphe dans le div

Un autre paragraphe dans le div

Un autre paragraphe

Texte écrit en JavaScript

157

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Ajouter, modifier ou supprimer des éléments du DOM avec JavaScript

2. Insérer un noeud dans le DOM

On peut utiliser la méthode **appendChild()** de l'interface **Node** pour ajouter un noeud en tant que dernier enfant d'un parent, mais elle ne peut ajouter qu'un seul noeud à la fois et retourne l'objet ajouté. En revanche, la méthode **append()** de **ParentNode** peut ajouter plusieurs noeuds ou des chaînes de caractères, et elle n'a pas de valeur de retour.

```
JS cours.js > ...
1 let b = document.body;
2 let newP = document.createElement('p');
3 let newTexte = document.createTextNode('Texte inséré avec appendChild()');
4
5 newP.textContent = 'Paragraphe créé et inséré grâce au JavaScript';
6
7 //Ceci fonctionne
8 b.append(newP, 'Texte inséré avec append()');
9
10 //Ceci fonctionne
11 b.appendChild(newTexte);
12
13 //Ceci ne fonctionne pas :
14 b.appendChild('Texte écrit en JavaScript');
15 //Ceci ne fonctionne pas non plus :
16 b.appendChild(newP, newTexte);
17
```

158

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Ajouter, modifier ou supprimer des éléments du DOM avec JavaScript

2. Insérer un noeud dans le DOM

Si vous voulez insérer l'élément à un endroit spécifique dans le DOM, vous pouvez utiliser `insertBefore()`.

```
JS cours.js > ...
1 // Crée un nouveau titre <h2>
2 const nouveauTitre = document.createElement('h2');
3 nouveauTitre.textContent = 'Titre inséré avant un autre élément';
4
5 // Sélectionne un élément existant pour l'insertion
6 const premierParagraphe = document.querySelector('p');
7
8 // Insère le nouveau titre avant le premier paragraphe
9 body.insertBefore(nouveauTitre, premierParagraphe);
10
```

159

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Ajouter, modifier ou supprimer des éléments du DOM avec JavaScript

2. Insérer un noeud dans le DOM

Une autre façon d'insérer des éléments dans le DOM est d'utiliser `innerHTML`, qui permet d'ajouter du contenu HTML directement à l'intérieur d'un élément. Cependant, il faut être prudent avec cette méthode car elle peut entraîner des problèmes de sécurité (injection de code malveillant) si le contenu est manipulé de manière incorrecte.

```
JS cours.js
1 // Ajoute un nouveau paragraphe en utilisant innerHTML
2 body.innerHTML += '<p>Paragraphe ajouté avec innerHTML</p>';
3
```

160

3. Manipulation du DOM

3.2. DOM ou Document Object Model

-> Ajouter, modifier ou supprimer des éléments du DOM avec JavaScript

3. Supprimer un noeud du DOM

Pour supprimer totalement un noeud du DOM, on peut déjà utiliser la méthode `removeChild()` de `Node` qui va supprimer un noeud enfant passé en argument d'un certain noeud parent de l'arborescence du DOM et retourner le noeud retiré.

On peut également utiliser la méthode `remove()` du mixin `ChildNode()` qui permet tout simplement de retirer un noeud de l'arborescence et qui dispose aujourd'hui d'un bon support par les navigateurs (cette façon de faire est plus récente que la précédente).

```
JS cours.js > ...
1 let b = document.body;
2 let p1 = document.getElementById('p1');
3 let p2 = document.getElementById('p2');
4
5 //Supprime p1 du DOM et renvoie le noeud supprimé
6 let eltDel = b.removeChild(p1);
7
8 //Supprime p2 du DOM
9 p2.remove();
10
```

161

Exercice TP 6:

Scanner ou cliquer sur:

https://docs.google.com/document/d/1SOAWQEGt2vzM7oRVaE2x1ubbzJQ_M-_ulOg5MXkRMjo/edit?usp=sharing



162

3. Manipulation du DOM

3.2. DOM ou Document Object Model

Exemple :



163

4. Gestion des événements

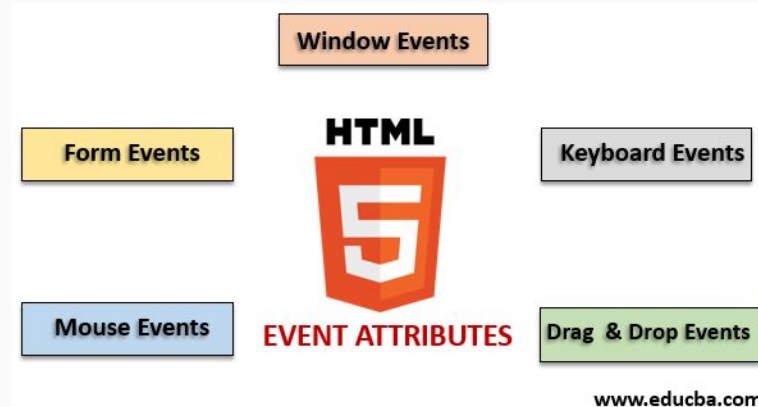
4.1. Définition

Un événement est une action effectuée soit par l'utilisateur, soit par le navigateur.

Il existe plusieurs types d'événements :

Caractéristiques principales :

- On peut « écouter » cet événement (le système nous notifié lorsqu'il survient, permettant ainsi de le détecter)
- On peut « réagir » à cet événement (associer du code qui s'exécute automatiquement lors de l'événement)



164

4. Gestion des événements

4.2. Programmation Événementielle

Concept :

La programmation événementielle consiste à associer une fonction à l'occurrence d'un événement sur un élément.

Fonctionnement :

La fonction est déclenchée (exécutée) lorsque l'événement se produit sur l'élément cible (target)

La fonction associée à un événement s'appelle « gestionnaire d'événement » (event handler) ou « écouteur d'événement » (event listener)

Les gestionnaires d'événements se divisent en deux parties :

Partie 1 : Écouter le déclenchement de l'événement

Partie 2 : Code à exécuter dès que l'événement se produit

4. Gestion des événements

4.3. Événements courants :

Événement	Déclencheur
click	L'utilisateur clique sur un élément
mouseover	La souris survole un élément
mouseout	La souris quitte un élément
keydown	L'utilisateur appuie sur une touche
keyup	L'utilisateur relâche une touche
submit	L'utilisateur soumet un formulaire
change	L'utilisateur change la valeur d'une entrée
load	La page se charge complètement
focus	Un élément reçoit le focus
blur	Un élément perd le focus

4. Gestion des événements

4.4. Trois façons d'implémenter un gestionnaire d'événement

Méthode 1 : Attributs HTML (Non recommandé)

L'utilisation d'attributs HTML pour gérer un événement est la méthode la plus ancienne.

Ces attributs HTML de type événement possèdent le nom de l'événement précédé par « on » :

Exemples :

onclick → événement "clic sur un élément"

onmouseover → événement "passage de la souris sur un élément"

onmouseout → événement "sortie de la souris d'un élément"

onkeydown → événement "touche enfoncée"

onsubmit → événement "soumission du formulaire"

4. Gestion des événements

4.4. Trois façons d'implémenter un gestionnaire d'événement

Exemple:

```
<button onclick="afficherMessage()">Cliquez-moi</button>

<script>
  function afficherMessage() {
    alert("Vous avez cliqué !");
  }
</script>
```


4. Gestion des événements

4.4. Trois façons d'implémenter un gestionnaire d'événement

Méthode 2 - Propriétés JavaScript

Chaque événement est représenté en JavaScript par un objet basé sur l'interface Event.

Les gestionnaires d'événements sont des propriétés nommées avec le préfixe « on » suivi du nom de l'événement :

- onclick
- onmouseover
- onmouseout
- onchange
- etc.

On associe généralement une fonction à ces propriétés pour exécuter du code lorsqu'un événement est déclenché.

169

4. Gestion des événements

4.4. Trois façons d'implémenter un gestionnaire d'événement

Exemple:

```
<button id="btn">Cliquez-moi</button>

<script>
  let element = document.getElementById("btn");

  element.onclick = function () {
    alert("Vous avez cliqué !");
  };
</script>
```

170

4. Gestion des événements

4.4. Trois façons d'implémenter un gestionnaire d'événement

Exemple:

```
<button id="btn">Cliquez-moi</button>

<script>
  let element = document.getElementById("btn");

  element.onclick = afficherMessage;
  function afficherMessage() {
    alert("Vous avez cliqué !");
  }
</script>
```

171

4. Gestion des événements

4.4. Trois façons d'implémenter un gestionnaire d'événement

Méthode 3 - addEventListener() (Recommandé)

La méthode **addEventListener()** appliquée sur un élément DOM lui ajoute un gestionnaire pour un événement particulier.

Syntaxe:

element.**addEventListener**(événement, fonction_de_rappel);

Paramètres :

- element : un élément HTML auquel l'événement est attaché
- événement : le nom de l'événement (sans le "on")
- fonction_de_rappel : la fonction qui va gérer l'événement

172

4. Gestion des événements

4.4. Trois façons d'implémenter un gestionnaire d'événement

Exemple:

```
<button id="btn">Cliquez-moi</button>

<script>
  let element = document.getElementById("btn");
  element.addEventListener("click", message);

  function message() {
    alert("Vous avez cliqué !");
  }
</script>
```

173

4. Gestion des événements

4.5. Supprimer un écouteur d'événement - removeEventListener()

La méthode `removeEventListener()` permet de supprimer un gestionnaire d'événement déclaré avec `addEventListener()`.

Syntaxe:

`element.removeEventListener(événement, fonction);`

Paramètres :

- Le type d'événement
- Le même nom de la fonction

174

4. Gestion des événements

4.6. preventDefault()

La méthode **preventDefault()** est utilisée pour empêcher le comportement par défaut d'un événement.

Concept clé :

Certains éléments HTML ont des comportements par défaut que le navigateur exécute automatiquement. Par exemple :

- Un lien (<a>) navigue vers l'URL spécifiée
- Un formulaire recharge la page et envoie les données
- Un clic droit affiche le menu contextuel
- Une touche écrite dans un champ de texte apparaît
- Avec preventDefault(), nous pouvons désactiver ces comportements par défaut et exécuter notre propre code à la place.

175

4. Gestion des événements

4.6. preventDefault()

Exemple:

```
let formulaire = document.getElementById('monFormulaire');
let resultat = document.getElementById('resultat');
let nom = document.getElementById('nom');

formulaire.addEventListener('submit', function (event) {
    event.preventDefault(); // Empêche le rechargement

    // Notre code personnalisé
    resultat.textContent = "Bienvenue, " + nom.value + " !";
    resultat.style.color = 'green';
});
```

176

4. Gestion des événements

Exercice

Développer une application permettant d'ajouter et de supprimer des tâches.

```
<input type="text" id="tache" placeholder="Entrez une tâche">
<button id="ajouterTache">Ajouter</button>
<ul id="listeTaches"></ul>
```

177

Exercice TP7:

Scanner ou cliquer sur:

<https://docs.google.com/document/d/1vWpd0uqWRfu4tqiZsAS9wkJvv-uk-gVG7o7AY0AjAv4/edit?usp=sharing>



178

5. Fonctions avancées

5.1. Les paramètres du reste (...rest)

-> Les paramètres du reste permettent de stocker une liste d'arguments dans un tableau qu'on va ensuite pouvoir manipuler.

```
let nom = 'Giraud', prenom = 'Pierre';
function profil(nom, prenom, ...hobbies){
    let h = '';
    for(hobbie of hobbies){
        h += hobbie + ', ';
    }
    console.log('Nom : ' + nom + 'Prénom : ' + prenom + 'Hobbies : ' + h);
}
```

179

5. Fonctions avancées

5.1. Les paramètres du reste (...rest)

```
let nom = 'Giraud', prenom = 'Pierre';
function profil(nom, prenom, ...hobbies){
    let h = '';
    for(hobbie of hobbies){
        h += hobbie + ', ';
    }
    console.log('Nom : ' + nom + 'Prénom : ' + prenom + 'Hobbies : ' + h);
}
```

```
profil('Giraud', 'Pierre');
profil('Giraud', 'Pierre', 'Trail');
profil('Giraud', 'Pierre', 'Trail', 'Triathlon');
```

```
[Running] node "d:\cours-javascript\cours.js"
```

```
Nom : GiraudPrénom : PierreHobbies :
```

```
Nom : GiraudPrénom : PierreHobbies : Trail,
```

```
Nom : GiraudPrénom : PierreHobbies : Trail, Triathlon,
```

```
[Done] exited with code=0 in 0.19 seconds
```

180

5. Fonctions avancées

5.2. L'opérateur de décomposition (...spread)

-> L'opérateur de décomposition permet de faire l'opération inverse, à savoir transformer un tableau en une liste d'arguments qu'on va pouvoir passer à une fonction.

-> L'opérateur de décomposition utilise la même syntaxe avec ... que les paramètres du reste à la différence qu'on va faire suivre les trois points par le nom d'un tableau existant.

-> L'opérateur de décomposition va alors casser le tableau en une liste d'arguments qui vont pouvoir être utilisés par la fonction.

```
let tb1 = [3, 5, 1, 32];  
let tb2 = [64, -5, 17];  
  
console.log('Plus grand nombre de tb1 : ' + Math.max(...tb1));  
console.log('Plus grand nombre de tb1 et tb2 : ' + Math.max(...tb1, ...tb2) );
```

181

5. Fonctions avancées

5.3. Les fonctions en JavaScript

Pour créer une fonction en JavaScript, on peut utiliser différentes syntaxes.

- ❖ une déclaration de fonction ;
- ❖ une expression de fonction ;
- ❖ une fonction fléchée ;

182

5. Fonctions avancées

5.3. Les fonctions en JavaScript

❖ une déclaration de fonction ;

Jusqu'à présent, nous avons principalement utilisé des déclarations de fonctions.

La syntaxe de déclaration de fonction est la suivante :

```
function disBonjour(){
    console.log('Bonjour');
}

disBonjour();
```

183

5. Fonctions avancées

5.3. Les fonctions en JavaScript

❖ une expression de fonction ;

Pour créer une expression de fonction, nous allons utiliser une syntaxe similaire à celle de déclaration, à la différence qu'on va cette fois-ci directement assigner notre fonction à une variable dont on choisira le nom.

```
let disBonjour= function(){
    console.log('Bonjour');
};

disBonjour();
```

184

5. Fonctions avancées

5.3. Les fonctions en JavaScript

❖ une expression de fonction ;

Généralement, lorsqu'on crée une fonction de cette manière, on utilise une **fonction anonyme** qu'on assigne ensuite à une variable. Pour appeler une fonction créée comme cela, on va pouvoir utiliser la variable comme une fonction, c'est-à-dire avec un couple de parenthèses après son nom.

185

5. Fonctions avancées

5.3. Les fonctions en JavaScript

❖ une expression de fonction ;

Les fonctions anonymes :

Une fonction anonyme est une fonction qui n'a pas de nom explicite. Ces fonctions sont souvent utilisées lorsqu'on a besoin de définir des comportements simples et temporaires, sans avoir besoin de réutiliser cette fonction ailleurs dans le code. Les fonctions anonymes sont souvent utilisées comme argument à d'autres fonctions, qui les appellent plus tard.

```
// Utilisation d'une fonction anonyme comme callback
setTimeout(function() {
    console.log("Ce message s'affiche après 2 secondes.");
}, 2000);
```

186

5. Fonctions avancées

5.3. Les fonctions en JavaScript

Déclarations de fonctions vs expressions de fonctions

```
disBonjour(); //Ceci fonctionne

function disBonjour(){
  console.log('Bonjour');
}

disAuRevoir(); //Ceci ne fonctionne pas

let disAuRevoir = function(){
  console.log('Au revoir');
}
```

187

5. Fonctions avancées

5.3. Les fonctions en JavaScript

❖ Les expressions de fonctions fléchées : syntaxe et intérêts

En ES6, une version plus concise des fonctions anonymes a été introduite : les fonctions fléchées.

```
let somme = (a, b) => a + b;
```

```
console.log(somme(1, 2));
```

188

5. Fonctions avancées

5.4. Gestion du délai d'exécution en JavaScript

La fonction prédéfinie : **setTimeout()**

La méthode `setTimeout()` permet d'exécuter une fonction ou un bloc de code après une certaine période définie (à la fin de ce qu'on appelle un « timer »).

```
// Affiche un message après 3 secondes (3000 millisecondes)
setTimeout(function() {
    console.log("Ce message apparaît après 3 secondes.");
}, 3000);
```

189

5. Fonctions avancées

5.4. Gestion du délai d'exécution en JavaScript

La fonction prédéfinie : **setInterval()**

La méthode `setInterval()` permet d'exécuter une fonction ou un bloc de code en l'appelant en boucle selon un intervalle de temps fixe entre chaque appel.

```
// Affiche un message toutes les 2 secondes (2000 millisecondes)
setInterval(function() {
    console.log("Ce message apparaît toutes les 2 secondes.");
}, 2000);
```

190

5. Fonctions avancées

5.4. Gestion du délai d'exécution en JavaScript

La fonction prédéfinie : `clearInterval()`

La méthode `clearInterval()` permet d'annuler l'exécution en boucle d'une fonction ou d'un bloc de code définie avec `setInterval()`.

```
// Déclare une variable pour stocker l'identifiant de l'intervalle
const intervalID = setInterval(function() {
    console.log("Ce message apparaît toutes les 2 secondes.");
}, 2000);

// Arrête l'intervalle après 10 secondes
setTimeout(function() {
    clearInterval(intervalID); // Arrête l'exécution répétée
    console.log("L'intervalle a été arrêté.");
}, 10000); // 10 000 millisecondes = 10 secondes
```

191

5. Fonctions avancées

Exercice 1

```
// TODO : compléter la fonction
function calculMoyenne(matiere, ...notes) {
    // 1. Si aucune note n'est fournie, retourner "Aucune note pour MATIERE"
    // 2. Calculer la somme des notes
    // 3. Calculer la moyenne
    // 4. Retourner le message final
}

// Quelques appels de test
console.log(calculMoyenne("JS", 10, 14, 16)); // La moyenne en JS est 13.33
console.log(calculMoyenne("Algèbre", 8, 12)); // La moyenne en Algèbre est 10
console.log(calculMoyenne("P00")); // Aucune note pour P00
```

192

5. Fonctions avancées

Exercice 2

```
const a = [1, 2];  
const b = [3, 4];  
  
// TODO : créer un nouveau tableau 'tous' avec spread  
// const tous = ...  
  
console.log(tous);  
// Résultat attendu : [1, 2, 3, 4]
```

193

5. Fonctions avancées

Exercice 3

```
const nombres = [2, 4, 6];  
  
// TODO : Complétez la ligne suivante avec une fonction fléchée  
const carres = nombres.map(/* VOTRE CODE ICI */);  
  
console.log(carres); // Doit afficher [4, 16, 36]
```

194